

Maximizing Match Quality in Online Skill-Based Routing

ANONYMOUS AUTHOR(S)

We study online task assignment with reusable resources, arising in peer reviewer assignment, skill-based call routing, rideshare matching, emergency dispatch, and crowdsourcing: Tasks arrive according to a geometric process and must be immediately assigned to servers that become temporarily busy, yielding rewards that depend on the task-server pair. The goal is to maximize long-run average reward.

We cast the problem as a contextual restless multi-armed bandit (CRMAB). We derive an occupancy-measure LP (OCC-LP) that upper-bounds the reward of any work-conserving policy. From its dual we obtain LAG, a closed-form index policy. We introduce an (m, n) -scaling regime that yields the first deterministic fluid limit for a contextual bandit. In this regime, LAG is asymptotically optimal.

Finally, we evaluate our algorithms on peer review datasets and call center routing. The Lagrangian index achieves 83–99% of the LP upper bound across all load levels. Moreover, the Lagrangian index policy significantly outperforms the natural greedy assignment scheme, improving reward by up to 12% on journal data and 36% on structured synthetic instances.

Additional Key Words and Phrases: contextual restless multi-armed bandits, online resource allocation, reusable resources, M/G/R/R loss systems, skill-based routing, mean-field optimality, occupancy-measure linear programming

ACM Reference Format:

Anonymous Author(s). 2027. Maximizing Match Quality in Online Skill-Based Routing. In *Proceedings of ACM SIGMETRICS (SIGMETRICS '27)*. ACM, New York, NY, USA, 28 pages.

1 Introduction

Many service systems maintain a pool of heterogeneous reusable resources — agents, servers, responders — that must be assigned to sequentially arriving jobs. In a skill-based call center, an incoming call must be routed to one of R available agent pools; in a data center, an arriving job is dispatched to one of R server types whose architecture may or may not suit its workload; in emergency dispatch, an incident is assigned k units from a heterogeneous fleet. A large body of literature in the queueing and performance evaluation communities has studied these skill-based parallel service systems under binary compatibility: Server r can serve job type v , or it cannot [1, 2, 26]. Routing decisions in these models are constrained by the compatibility graph but are otherwise interchangeable: Any compatible assignment is as good as any other.

Real systems rarely satisfy this equivalence. In a call center, agents differ not just in which call types they can handle but also in how effectively they serve them: A specialist consistently outperforms a generalist on calls both are qualified for. In a data center, a job may run significantly faster on a server whose architecture suits its workload. In peer review, the quality of a referee report depends on how closely a reviewer's expertise matches the paper's topic. In each of these settings, the assignment decision yields a quality score s_{vr} , a measure of how well resource r serves task type v . The system operator's goal is to maximize the total quality of assignments over time.

We study a discrete-time generalization of the skill-based parallel service model in which the binary compatibility graph is replaced by a real-valued score matrix $S \in \mathbb{R}^{V \times R}$. At each time step, a task of type v arrives i.i.d.; it must be immediately assigned to k currently available servers, each of which then becomes busy for a random bounded number of steps D_v before returning to the pool. The objective is to maximize long-run average reward per step, where assigning server r to a type- v task yields reward s_{vr} . Tasks arriving when fewer than k servers are free are dropped.

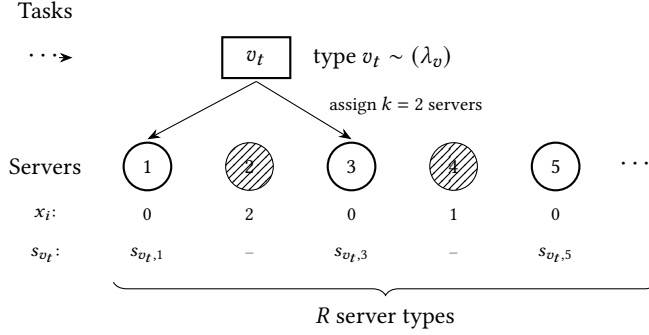


Fig. 1. Online assignment with reusable servers. At each step t , a task of type v_t arrives i.i.d. with $\Pr[v_t = v] = \lambda_v$ and is assigned to k available servers. State x_i denotes steps until server i is free; $x_i = 0$ means available. Scores $s_{v_t, r}$ are shown for available servers only; hatched circles are busy.

1.1 Model

We consider a pool of R servers and a stream of tasks arriving in discrete time $t = 1, 2, \dots$. We are given a match score matrix $S \in \mathbb{R}^{V \times R}$, where $s_{vr} \in \mathbb{R}$ is the reward for assigning server r to a type- v task.

In the reviewer-assignment setting, tasks correspond to submitted papers and resources correspond to reviewers; the reward s_{vr} represents the suitability or expertise match of reviewer r for paper of type v_t . In skill-based routing for call-centers, tasks corresponding to incoming calls and resources correspond to customer service agents. Here, s_{vr} represents the suitability of agent r to serve a call of type v .

Encompassing these and similar problems, our model is a discrete-time M/G/R/R system specified by the following assumptions.

- (A1) Discrete-time arrivals (λ_v). At each step at most one task arrives: A task of type $v \in [V]$ arrives with probability $\lambda_v \geq 0$, independently across steps, and no task arrives with probability $1 - \sum_{v \in [V]} \lambda_v$. We call λ_v the type- v arrival rate.
- (A2) Type-dependent service (D_v). When a type- v task is assigned, a single service duration D_v is drawn i.i.d. from a distribution that depends only on the task type v . All k assigned servers are busy for those D_v steps and return to the pool together. The support is bounded, $D_v \leq d$.
- (A3) All-or-nothing blocking (M/G/R/R). At a type- v arrival, if at least k servers are free the policy assigns a set $\mathcal{A}_t$ of exactly k of them, yielding reward $\sum_{r \in \mathcal{A}_t} s_{vtr}$; if fewer than k are free the task is dropped and leaves the system.
- (A4) Work-conserving policies. We consider only work-conserving policies: Whenever at least k servers are free at a type- v arrival, the policy assigns k of them. A policy chooses which servers to assign, but may not leave a task unserved when it could be served.

The objective is to maximize long-run average reward per step:

$$\text{maximize} \quad \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \sum_{r \in \mathcal{A}_t} s_{vtr}. \quad (1)$$

1.2 Why This Problem Is Hard

A natural baseline for our problem is a greedy assignment policy: Upon arrival of a type- v task, assign the k currently available servers with the highest scores for type v . Greedy assignment fails to account for future demand: A server that scores well across many task types will be claimed by nearly every arrival and be unavailable when a task arrives for which it is the best match. More precisely, the value of assigning server r to the current task depends not only on the immediate reward s_{vr} , but on the expected reward forgone by future tasks that arrive while r is occupied. Despite its limitations, greedy is the dominant policy in practice. Automated reviewer assignment systems rank available reviewers by expertise score and assign the top k [30]. In call centers, the routing component of the Wallace and Whitt [26] staffing policy dispatches each call to the best-available agent for its type.

Optimal control can be formulated as an MDP over the joint availability state of all servers, but the state space is exponential in the number of servers, making exact dynamic programming intractable at any practical scale. We instead formulate our problem using a restless bandit framework and derive an occupancy-measure based LP, OCC-LP. The dual of OCC-LP decomposes the problem into independent per-server subproblems, yielding a closed-form index policy.

1.3 Our Contributions

We develop LAG, an index policy for online assignment of reusable, quality-differentiated resources, and show it is near-optimal both in theory and in practice. LAG is asymptotically optimal in a mean-field limit, and on real peer-review traces and synthetic instances it attains 83–99% of a tight LP upper bound—including well outside the regime where our optimality theory applies—improving over the natural greedy policy by up to 12% on peer-review datasets. We obtain LAG by viewing the problem through a restless-bandit lens: each server is an arm and the arriving task type is a context, making the system a contextual restless multi-armed bandit (CRMAB) to which we can adapt restless-bandit index machinery [25, 27, 28].

A tight LP bound and the LAG policy. LP relaxations are a standard tool for online-matching problems as well as restless bandit problems, and our occupancy-measure LP (OCC-LP) is in this tradition. Our OCC-LP is a strictly tighter variant of the KIID LP of Dickerson et al. [10]. The dual of OCC-LP decouples the assignment problem across servers, and each per-server subproblem is solved by a threshold policy. This yields LAG: a scalar index $L(v, r)$ per (task type, server) pair, computed once from the LP solution, after which each arrival is assigned greedily to its highest-index available servers.

A new mean-field scaling regime. Proving optimality for a *contextual* restless bandit requires a scaling that existing analyses do not provide: the classical n -only scaling of Weber and Weiss [27] keeps one random arrival per step, so the fluid system never settles to a deterministic fixed point. We introduce an (m, n) -scaling in which n copies of each server and m i.i.d. arrivals grow together; scaling m concentrates the empirical context distribution each step, restoring the deterministic fluid limit needed for a fixed-point analysis. In this regime we prove LAG is asymptotically optimal:

THEOREM 1.1 (INFORMAL; SEE THEOREM 4.1). *In the (m, n) -scaled system, under a mild non-degeneracy condition,*

$$\frac{\text{Reward}_{m,n}(\text{LAG})}{mn} \rightarrow \text{OPT}(\text{OCC-LP}(0)) \quad \text{as } m, n \rightarrow \infty.$$

Empirical performance. On real peer-review datasets and structured synthetic instances, LAG attains 83–99% of the OCC-LP upper bound across all load levels—far outside the mean-field

regime—and improves over greedy assignment by up to 12% on peer-review datasets and up to 36% on synthetic instances. We also validate convergence to the mean-field limit and characterize how the gap between LAG and greedy grows with score heterogeneity.

2 Related Work

Two research communities have studied problems related to ours. The queueing literature on parallel server systems (§2.1) asks how to route and staff heterogeneous servers efficiently; the online matching literature (§2.2) studies assignment of reusable resources to arriving tasks and evaluates algorithms by competitive ratio. We discuss how each community has approached these problems and where our model and results depart from theirs. We also discuss the restless multi-armed bandit literature (§2.3), which we draw on for both the CRMAB formulation of our problem and the fluid-scaling proof techniques behind our main result.

2.1 Parallel Server Systems

The parallel service literature studies systems in which heterogeneous servers must be matched to a stream of arriving tasks subject to compatibility constraints. Call centers, where agents with different skills are routed to different call types, are the canonical motivating application. We highlight two threads most relevant to our work: Routing policy design and staffing.

Routing. Several papers ask which routing policy optimizes system performance, typically measured by throughput or delay. Adan and Weiss [1] analyze the FCFS-ALIS policy under binary compatibility and derive exact product-form stationary distributions for the joint occupancy state. Armony [3] studies a single customer class routed to server pools of differing speeds and shows that Fastest-Servers-First asymptotically minimizes queue length in the Halfin-Whitt regime. Mandelbaum and Stolyar [17] consider flexible multi-skilled servers with convex delay costs; their service rates μ_{ij} vary by both customer type and server, and the generalized $c\mu$ -rule (which routes each customer to the server maximizing the instantaneous rate of cost decrease) is asymptotically optimal in heavy traffic. Atar [4] proves related results for multi-class systems via a diffusion control approach. In all of these, service heterogeneity enters as a capacity or cost parameter; the optimization objective is throughput or delay, not match quality.

Staffing. Wallace and Whitt [26] ask how many agents of each skill type a call center should employ to meet per-class service targets. Their three-stage algorithm combines Erlang-based total headcount, square-root staffing for primary skills, and fair assignment of secondary skills, and requires that service times not depend on the call type or the agent.

All of the above treat compatible assignments as interchangeable in quality. We replace the binary compatibility graph with a real-valued score matrix $S \in \mathbb{R}^{V \times R}$, where s_{vr} is the reward for assigning server type r to a type- v task, and optimize long-run average reward rather than throughput or delay. The OCC-LP is the natural analogue of the throughput LP used throughout this literature; LAG is the index policy derived from its dual.

2.2 Online Assignment with Reusable Resources

Online bipartite matching with reusable resources is a well-studied problem [14]: agents matched to an arriving task become temporarily unavailable, returning to the pool after a random occupation time, capturing ridesharing, crowdsourcing, organ donation, and server allocation. The dominant performance criterion is competitive ratio—an algorithm’s expected reward guaranteed as a fraction of the expected reward of an offline oracle that knows all future arrivals. Dickerson et al. [10] study the OM-RR-KAD model (Online Matching with Reusable Resources under a Known Adversarial Distribution) and derive an LP-based algorithm (ADAP) achieving competitive ratio $\frac{1}{2} - \varepsilon$ for

any $\varepsilon > 0$; they also establish that no non-adaptive algorithm can achieve better than $\frac{1}{2} + o(1)$. Sumita et al. [24] extend this to multiple agents per task and agent rejections. Shanks, Yu and Jacobson [22] study a variant with rank-1 similarity scores; Delong et al. [9] achieve ≈ 0.589 in the vertex-weighted adversarial case.

Rather than targeting a fractional guarantee relative to an offline oracle, our work directly maximizes expected long-run average reward in the stationary i.i.d. setting: the OCC-LP tightly characterizes the achievable long-run average reward rate, and LAG is asymptotically optimal with respect to this benchmark in the mean-field limit. This approach achieves reward approaching 100% of the OCC-LP bound as the system scales, compared to the $\frac{1}{2}$ ceiling that bounds competitive ratio performance in this literature. Empirically, LAG consistently achieves 83–99% of the OCC-LP bound on application traces—well above what competitive ratio algorithms achieve in practice. Our OCC-LP is related to, but strictly tighter than, the benchmark LP of [10]. The algorithm of [10] achieving the competitive ratio guarantee requires running simulation at each timestep and is not practical to deploy; we include their ALG-SC-LP heuristic as an experimental baseline in Section 5 and show that LAG consistently outperforms it.

2.3 Restless Bandits and Mean-Field Analysis

A restless multi-armed bandit (RMAB) [28] consists of n arms, each an independent Markov chain. At each step, the decision-maker selects exactly k arms to *activate* (the per-step budget); each arm receives a reward and transitions according to its active or passive dynamics, and the objective is to maximize long-run average reward subject to the per-step budget constraint. Optimal RMAB control is PSPACE-hard [20], which has motivated a rich literature on tractable index heuristics with theoretical guarantees [19].

Whittle [28] proposed relaxing the per-step budget to a time-average constraint via Lagrangian duality, which decouples the arms into n independent single-arm subproblems. When each arm is *indexable*—meaning the set of states preferring passivity grows monotonically in the Lagrange multiplier—the break-even multiplier defines a scalar Whittle index for each (arm, state) pair, and the resulting policy activates the k arms with the highest current indices. Weber and Weiss [27] proved this policy asymptotically optimal as $k, n \rightarrow \infty$ with the activation fraction k/n fixed, under the assumption that the fluid ODE describing the mean-field dynamics has a unique globally stable equilibrium. Their proof establishes that the stochastic occupancy measure concentrates on this equilibrium as $n \rightarrow \infty$, and that the equilibrium coincides with the LP-relaxation optimum. Verloop [25] substantially extended these results to multi-class RMAB problems with heterogeneous arms, dynamic arm arrivals and departures, and multiple simultaneous actions, using a combination of fluid-scaling techniques and LP analysis; our proof of Theorem 4.1 follows Verloop’s approach directly. Avrachenkov, Borkar and Shah [5] study a Lagrangian index policy for average-reward RMABs and prove asymptotic optimality in the homogeneous-arm setting.

A *contextual* RMAB (CRMAB) extends this framework: at each step a context is observed that modulates arm rewards and, possibly, arm transitions. Our problem is naturally a CRMAB: each server type r is a bandit arm and the arriving task type v_t is the context, determining both the reward and the occupation time for each arm. Chen and Hou [8] introduce a CRMAB in which the context follows a Markov process, arm transitions depend on the context, and the per-step budget is context-dependent; they derive a Lagrangian index policy via dual decomposition, but demonstrate asymptotic optimality only numerically. Our work proves asymptotic optimality of LAG in a new (m, n) -scaling regime, in which both the number of simultaneous arrivals m and the number of resource copies n grow (Section 4). This regime is necessary: in the standard $n \rightarrow \infty$ scaling with one arrival per step, the per-step context is random, which prevents the fluid system from converging to a deterministic fixed point, and the Lagrangian reward may not reach the

fluid LP bound. Scaling m alongside n allows the empirical context distribution to concentrate at each step by the law of large numbers, making the fluid limit deterministic and amenable to the fixed-point analysis of [25].

3 Our Proposed Policy

We model the problem as a contextual restless multi-armed bandit (CRMAB). Each server is an arm, and the type of the task arriving at each step is the context. The CRMAB is exactly the model of Section 1.1, read one arm at a time.

Arm. Each server $r \in [R]$ is an arm. At step t , arm r has state $X_r(t) \in \{0, 1, \dots, d\}$, where $X_r(t) = 0$ means r is free and $X_r(t) = j > 0$ means r has j steps of service remaining; here d bounds the service duration ($D_v \leq d$).

Context. At each step t , a task of type $v_t \in [V]$ arrives i.i.d. with $\Pr[v_t = v] = \lambda_v$, and no task arrives with probability $1 - \sum_v \lambda_v$, independently of the arm states.

Action. When at least k servers are free, a policy selects an assignment set $\mathcal{A}_t \subseteq \{r \in [R] : X_r(t) = 0\}$ with $|\mathcal{A}_t| = k$; when fewer than k are free the task is dropped ($\mathcal{A}_t = \emptyset$). We consider only work-conserving policies, which assign whenever k servers are free.

Transitions. An assigned server becomes busy for the task's service duration, and every other busy server has one step of service elapse. That is, for each $r \in [R]$,

$$X_r(t+1) = \begin{cases} D_{v_t} & r \in \mathcal{A}_t, \\ (X_r(t) - 1)^+ & r \notin \mathcal{A}_t, \end{cases}$$

where D_{v_t} is a single duration drawn at the moment of assignment and shared by all k assigned servers, so they free together.

Reward. Assigning $\mathcal{A}_t$ to a type- v_t task yields reward $\sum_{r \in \mathcal{A}_t} s_{v_t r}$. The objective is to maximize (1).

3.1 OCC-LP: The Occupancy-Measure LP Upper Bound

The *occupancy-measure LP* (OCC-LP) is a linear program over the long-run assignment rates of a work-conserving policy; its optimum upper-bounds the reward of any such policy, as we prove in Proposition 3.2. The decision variables are $u_{vr} \geq 0$ for $v \in [V]$, $r \in [R]$, the long-run rate of assignments of server r to type- v tasks (assignments per step), and $\phi_r \geq 0$ for $r \in [R]$, the steady-state free fraction of server r . One further quantity enters the LP as a constant: The *blocking probability* B , the long-run fraction of arrivals dropped because fewer than k servers are free. By Lemma 3.1, B is the same for every work-conserving policy and every task type.

$$\max_{u, \phi} \sum_{v \in [V], r \in [R]} s_{vr} u_{vr} \quad (\text{OCC-LP})$$

$$\text{s.t.} \quad \sum_{r \in [R]} u_{vr} = k \lambda_v (1 - B) \quad \forall v \in [V] \quad (2)$$

$$\phi_r = 1 - \sum_{v \in [V]} \mathbb{E}[D_v] u_{vr} \quad \forall r \in [R] \quad (3)$$

$$0 \leq u_{vr} \leq \lambda_v \phi_r \quad \forall v \in [V], r \in [R] \quad (4)$$

Constraint (2) fixes the type- v assignment rate: Type- v tasks arrive at rate λ_v , a fraction $1 - B$ of them are served, and each served task occupies exactly k servers, giving rate $k \lambda_v (1 - B)$. Constraint (3) is Little's Law: Each assignment of type v to server r occupies it for expected duration $\mathbb{E}[D_v]$ at rate u_{vr} , so the total busy fraction is $\sum_v \mathbb{E}[D_v] u_{vr}$. Constraint (4) ensures the assignment rate u_{vr}

Table 1. Summary of notation used in Section 3.

Symbol	Description
<i>Problem parameters</i>	
R	Number of servers
V	Number of task types
$\lambda_v \geq 0$	Arrival rate of a type- v task (probability per step; $\sum_v \lambda_v \leq 1$)
k	Number of servers assigned per type- v task
$s_{vr} \in \mathbb{R}$	Match reward for assigning server r to a type- v task
$D_v \leq d$	Service duration of a type- v task, shared by its k servers (random integer)
<i>CRMAB model</i>	
$X_r(t) \in \{0, \dots, d\}$	State of server r at step t ; $X_r(t) = 0$ means free
$\mathcal{A}_t \subseteq [R]$	Set of k servers assigned at step t
<i>OCC-LP variables</i>	
$u_{vr} \geq 0$	Long-run assignment rate of server r to type- v tasks (per step)
$\phi_r \geq 0$	Steady-state free fraction of server r
<i>Lagrangian index</i>	
ℓ_v^*	Dual variable for the type- v demand constraint (2)
q_r^*	Dual variable (Arm- r LP value) for the capacity constraint (3)
$L(v, r) = s_{vr} - \ell_v^* - \mathbb{E}[D_v] q_r^*$	Lagrangian index of server r for task type v

does not exceed the rate at which server r is simultaneously free and a type- v task arrives; by independence of arrivals and server state in the fluid limit, this rate is $\lambda_v \phi_r$.

LEMMA 3.1. *Let N_t denote the number of busy servers at step t . Under any work-conserving policy the process (N_t) undergoes the same state transitions, and the steady-state blocking probability $B := \lim_{t \rightarrow \infty} \Pr[N_t \geq R - k + 1]$ is independent of both the policy and the arriving task type.*

PROOF. Run any two work-conserving policies on the same arrival sequence and the same service time draws, starting from $N_0 = 0$. Suppose N_t agrees under both. A work-conserving policy assigns exactly k servers iff $N_t \leq R - k$, so under the common value of N_t both policies make the same accept-or-drop decision. On acceptance both occupy k servers for the same duration D_v , which depends only on the type v_t and not on which servers are chosen, so the same number become busy and the same number later free; hence N_{t+1} again agrees. By induction the two processes coincide step by step, so they share the same transitions and the same steady-state blocking probability B . For type-independence, v_t is i.i.d. and independent of the history, hence of N_t , while the event $\{N_t \geq R - k + 1\}$ is a function of N_t alone (as k is constant); thus $\Pr[\text{block} \mid v_t = v] = \Pr[N_t \geq R - k + 1]$ for every v . $\square$

REMARK 1. *In continuous time the blocking probability has a closed form. Modelling each task as a call that holds k of R circuits for one duration D_v gives an Erlang loss network [15], whose blocking depends only on the means: $B = \text{ErlangB}(R/k, \sum_v \lambda_v \mathbb{E}[D_v])$. Our discrete-time model admits no such closed form: B is sensitive to the service distribution beyond its mean. We therefore compute B numerically; the Erlang expression still serves as a useful reference value.*

We write OCC-LP(B) to make the blocking probability explicit, and OCC-LP(0) for the $B = 0$ instance in which the demand constraint reads $\sum_r u_{vr} = k \lambda_v$. As the system scales the blocking vanishes (Section 4), so OCC-LP(0) is the fluid limit from which we derive LAG and against which we prove its optimality, while OCC-LP(B) is the valid upper bound at finite scale.

PROPOSITION 3.2 (LP UPPER BOUND). For any work-conserving policy π , with B the blocking probability of Lemma 3.1,

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \sum_{r \in \mathcal{A}_t} s_{v_t r} \leq \text{OPT}(\text{OCC-LP}(B)).$$

PROOF. Fix a policy π . Let ϕ_r^π be the long-run fraction of steps server r is free, and u_{vr}^π the long-run rate of type- v assignments to server r :

$$\phi_r^\pi := \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbf{1}[X_r^\pi(t) = 0], \quad u_{vr}^\pi := \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbf{1}[v_t = v, r \in \mathcal{A}_t].$$

The objective equals $\sum_{v,r} s_{vr} u_{vr}^\pi$. We show (u^π, ϕ^π) is feasible for (OCC-LP).

Constraint (2). At each type- v arrival a work-conserving policy assigns exactly k servers when at least k are free, and drops the task otherwise. Let $T(v) := \#\{t \leq T : v_t = v\}$ and let $T_{\text{acc}}(v)$ count the accepted type- v arrivals. Then

$$\sum_{r \in [R]} u_{vr}^\pi = \lim_{T \rightarrow \infty} \frac{k T_{\text{acc}}(v)}{T} = k \lambda_v (1 - B),$$

using $T(v)/T \rightarrow \lambda_v$ and $T_{\text{acc}}(v)/T(v) \rightarrow 1 - B$ a.s., where B is the policy- and type-independent blocking probability of Lemma 3.1.

Constraint (3). By Little's Law applied to server r : The long-run assignment rate to server r is $\sum_v u_{vr}^\pi$, the mean service time per assignment is $\sum_v \mathbb{E}[D_v] u_{vr}^\pi / \sum_v u_{vr}^\pi$, and the average occupancy is $1 - \phi_r^\pi$. Hence $1 - \phi_r^\pi = \sum_v \mathbb{E}[D_v] u_{vr}^\pi$.

Constraint (4). Server r can only be assigned when free, so

$$u_{vr}^\pi \leq \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbf{1}[v_t = v, X_r^\pi(t) = 0].$$

Let $\mathcal{F}_t$ denote the history before step t (all arrivals, assignments, and service draws through step $t - 1$). The state $X_r^\pi(t)$ is $\mathcal{F}_t$ -measurable, while v_t is i.i.d. with $\Pr[v_t = v] = \lambda_v$ and independent of $\mathcal{F}_t$, so $M_T = \sum_{t=1}^T (\mathbf{1}[v_t = v] - \lambda_v) \mathbf{1}[X_r^\pi(t) = 0]$ is a martingale with bounded increments, giving $M_T/T \rightarrow 0$ a.s. Hence

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbf{1}[v_t = v, X_r^\pi(t) = 0] = \lambda_v \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbf{1}[X_r^\pi(t) = 0] = \lambda_v \phi_r^\pi.$$

Therefore (u^π, ϕ^π) is feasible for OCC-LP(B), giving $\sum_{v,r} s_{vr} u_{vr}^\pi \leq \text{OPT}(\text{OCC-LP}(B))$. $\square$

Note that this upper bound is similar to, but tighter than, the KIID variant of [10, LP (1)].

3.2 LAG, the Lagrangian Index Policy

We derive a Lagrangian index policy via dual decomposition of the OCC-LP. We show in Section 5 that it achieves 94–99% of the OCC-LP upper bound across all evaluated instances, and prove asymptotic optimality in Section 4.

Lagrangian decomposition. We derive the index from the fluid limit OCC-LP(0). Form its Lagrangian by relaxing the equality demand constraint (2) (with $B = 0$) using multipliers $\ell_v \in \mathbb{R}$, one per task type $v \in [V]$:

$$\begin{aligned}\mathcal{L}(u, \ell) &= \sum_{v \in [V], r \in [R]} s_{vr} u_{vr} + \sum_{v \in [V]} \ell_v \left(k \lambda_v - \sum_{r \in [R]} u_{vr} \right) \\ &= \sum_{r \in [R]} \sum_{v \in [V]} (s_{vr} - \ell_v) u_{vr} + \sum_{v \in [V]} \ell_v k \lambda_v,\end{aligned}$$

subject to (3) and (4). For fixed ℓ , the maximization over u decouples across servers $r \in [R]$. For each $r \in [R]$, the Arm- r LP is:

$$\begin{aligned}\max_{(u_{vr})_{v \in [V]}} \quad & \sum_{v \in [V]} (s_{vr} - \ell_v) u_{vr} & (\text{Arm-}r \text{ LP}) \\ \text{s.t.} \quad & 0 \leq u_{vr} \leq \lambda_v \phi_r \quad \forall v \in [V] \\ & \phi_r = 1 - \sum_{v \in [V]} \mathbb{E}[D_v] u_{vr}.\end{aligned}$$

Let $q_r(\ell)$ denote the optimal value of the Arm- r LP.

Dual LP. The Lagrangian dual problem is:

$$\min_{\ell \in \mathbb{R}^V} \sum_{r \in [R]} q_r(\ell) + \sum_{v \in [V]} \ell_v k \lambda_v.$$

By strong duality, the optimal dual value equals $\text{OPT}(\text{OCC-LP}(0))$. Let $\ell^* = (\ell_v^*)_{v \in [V]}$ be an optimal dual solution, (u^*, ϕ^*) the corresponding primal solution, and $q_r^* := q_r(\ell^*)$.

DEFINITION 3.1 (LAG POLICY). For each $v \in [V]$ and $r \in [R]$, define the Lagrangian index:

$$L(v, r) := s_{vr} - \ell_v^* - \mathbb{E}[D_v] q_r^*. \quad (5)$$

The LAG policy is: When a task of type $v \in [V]$ arrives, assign it to the k available servers with the largest values of $L(v, r)$, breaking ties arbitrarily.

The Lagrangian index is derived from the following principle: relax the constraint that exactly k servers are assigned to each arrival, and instead accept every (v, r) pair with $L(v, r) > 0$ whenever server r is free. This decoupled policy achieves time-average reward equal to $\text{OPT}(\text{OCC-LP}(0))$. The following proposition makes this precise.

PROPOSITION 3.3. The OCC-LP optimal solution (u^*, ϕ^*) satisfies, for every $v \in [V]$, $r \in [R]$:

$$\begin{aligned}L(v, r) > 0 &\implies u_{vr}^* = \lambda_v \phi_r^*, \\ L(v, r) = 0 &\implies u_{vr}^* \in [0, \lambda_v \phi_r^*], \\ L(v, r) < 0 &\implies u_{vr}^* = 0.\end{aligned}$$

In the full model, LAG assigns exactly k servers to each arrival, so this decoupled optimality result does not translate directly. In Section 4, we establish a mean-field scaling regime in which this distinction vanishes.

To prove Proposition 3.3, we translate the Arm- r LP into a bandit problem. Lemma 3.4 shows the optimal average reward of the bandit equals q_r^* . Lemma 3.5 shows the optimal bandit policy accepts type v in state $(0, v)$ if and only if $L(v, r) \geq 0$. Proposition 3.3 follows by combining the two lemmas. Proofs of Lemmas 3.4 and 3.5 are deferred to Appendix A.

DEFINITION 3.2 (ARM- r BANDIT). Given a penalty vector $\ell \in \mathbb{R}^V$, the Arm- r bandit is a single server observed in discrete time:

State. The server is either free ($X = 0$) or busy with $X \in \{1, \dots, d\}$ steps of service remaining. At each step a type- v task arrives with probability λ_v (and none with probability $1 - \sum_v \lambda_v$), independently of the server state.

Action. When the server is free and a type- v task arrives, a policy chooses to accept ($a = 1$) or reject ($a = 0$). Acceptance is only possible when the server is free.

Reward. Accepting a type- v task yields immediate reward $s_{vr} - \ell_v$; the server then becomes busy for D_v steps, drawn i.i.d. ($D_v \leq d$) and independently of all other quantities.

Objective. Let g_π denote the long-run average reward per step under policy π started from the free state. The time-average Arm- r bandit maximizes g_π over policies π .

The penalty ℓ_v^* is the shadow price of the demand constraint for type v : It reflects the opportunity cost of serving a type- v arrival, as determined by the global demand constraints (2).

LEMMA 3.4 (ARM- r BANDIT-LP EQUIVALENCE). The optimal average reward of the Arm- r bandit equals the value of the Arm- r LP: $g_* = q_r^*$.

LEMMA 3.5. The optimal policy for the time-average Arm- r bandit accepts type v in state $(0, v)$ if and only if:

$$\begin{aligned} s_{vr} - \ell_v > \mathbb{E}[D_v] q_r^* &\implies \pi^*(0, v) = 1, \\ s_{vr} - \ell_v = \mathbb{E}[D_v] q_r^* &\implies \pi^*(0, v) \in \{0, 1\}, \\ s_{vr} - \ell_v < \mathbb{E}[D_v] q_r^* &\implies \pi^*(0, v) = 0. \end{aligned}$$

Lemma 3.5 applied to the dual-optimal penalties ℓ^* tells us that the optimal Arm- r bandit policy accepts type v iff $L(v, r) > 0$ (with indifference at $L(v, r) = 0$). Lemma 3.4 tells us the occupancy measure of this policy equals (u^*, ϕ^*) , as shown in the achievability direction of Lemma 3.4. This establishes Proposition 3.3.

4 Mean-Field Optimality

In this section, we propose a mean-field scaling regime and prove that LAG is asymptotically optimal in the scaling regime.

Our analysis follows the Weber–Weiss tradition of mean-field proofs for index policies [25, 27], which rests on delicate fixed-point and convergence arguments for finite Markov chains. These arguments require each arm to have a finite state space, which the discrete-time model of Section 1.1 provides: with bounded service durations, each server’s state lies in $\{0, \dots, d\}$.

4.1 The Mean Field Scaling Regime

A rich body of prior work studies mean-field asymptotic optimality of index policies for restless bandit problems [16, 19, 25, 27]. The classical result of Weber and Weiss [27] shows that when there are n identical arms, the Whittle index policy is optimal as $n \rightarrow \infty$. However, these results do not directly extend to our *contextual* RMAB setting. It is not immediately clear how a CRMAB should be scaled. We begin by considering two natural approaches:

Batch-scaling (n-scaling). Suppose we scale the number of resources by a factor of n , so that there are n copies of each resource type. At each time step, a single arrival v_t is assigned nk resources out of a pool of size nR . This model is depicted in Figure 2b. Chen and Hou [8] consider a related scaling regime for general CRMABs.

The difficulty is that the arrival process remains stochastic at unit rate. As a result, even as $n \rightarrow \infty$, the system does not converge to a deterministic fixed point. Instead, the evolution of the

system depends on the realized sequence of contexts, and the fluid limit may exhibit non-trivial dynamics (e.g., limit cycles) driven by the context process.

Buffer-scaling (m -scaling). Alternatively, suppose there are m i.i.d. arrivals at each time step, and m copies of each resource type. Each arrival must be matched to k free resources, thus a total of mk assignments are made at each step. This model is depicted in Figure 2c.

This avoids the limit-cycle issue, since averaging over many arrivals produces a deterministic system. However, this scaling fundamentally changes the problem. At each time step, assignments can be made jointly across all arrivals, allowing resources to effectively choose the most favorable matches among a batch of tasks. This is strictly more powerful than in the original problem, where each resource only observes a single arrival.

Formally, under this scaling, the induced occupancy measures (u^π, ϕ^π) may violate the constraint (4) in OCC-LP. The limiting LP (e.g., the KIID LP [10]) therefore provides a looser upper bound. In our experiments, heuristics based on this LP can perform worse, likely because the relaxation models the problem less tightly. This is discussed in Section 5.

Thus, neither scaling provides the right generalization of the Weber–Weiss regime: n -scaling preserves the problem structure but does not yield a deterministic limit, while m -scaling yields a deterministic limit but relaxes the problem.

The (m, n) -scaling. We propose a scaling that combines both ideas. We scale the number of arrivals and resources, while ensuring that each resource only observes one arrival per time step.

At each time step, m i.i.d. arrivals occur, and there are mn resources of each type. For each resource type, the mn resources are uniformly partitioned into m groups of size n , and each group is assigned to one arrival. Thus, each arrival observes n resources of each type (for a total of nR), and must select nk of them. Assigned resources become unavailable for D_v steps (at most d).

We scale the arrival rate by m , the demand per arrival by n , and the resource pool by mn , so that the system load remains unchanged:

$$\rho = \frac{k}{R} \sum_v \lambda_v \mathbb{E}[D_v].$$

The load ρ is the long-run fraction of busy servers, the same for every work-conserving policy that serves all demand. Our optimality result requires the system to stay strictly underloaded, keeping enough free capacity to hold the largest task.

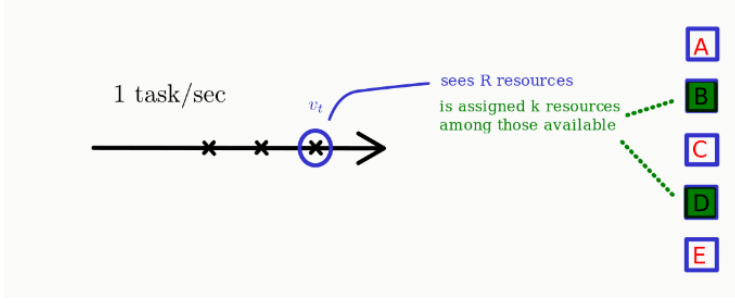
ASSUMPTION 4.1 (STRICT UNDERLOAD).

$$\rho < 1 - \frac{k}{R}.$$

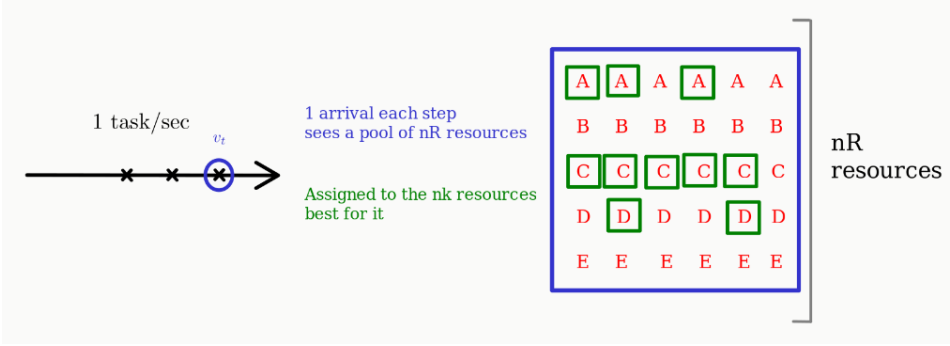
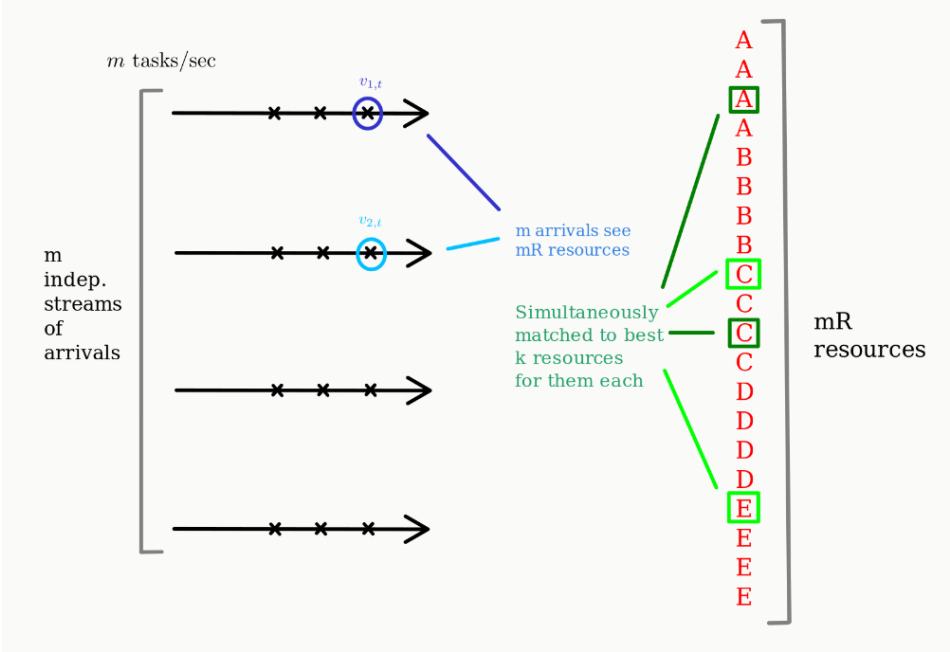
This is only slightly stronger than $\rho < 1$. For any feasible OCC-LP solution, the bound constraint (4) gives $\sum_r u_{vr} \leq \lambda_v \sum_r \phi_r$. The demand constraint (2) fixes the left side at $k\lambda_v$, so $k \leq \sum_r \phi_r$. Summing the capacity constraint (3) over r and using the demand constraint gives $\sum_r \phi_r = R - k \sum_v \lambda_v \mathbb{E}[D_v] = R(1 - \rho)$. Hence $k \leq R(1 - \rho)$, i.e., $\rho \leq 1 - k/R$. Assumption 4.1 asks for the strict inequality.

Under this scaling, the system admits a mean-field limit as $m, n \rightarrow \infty$. The randomness in arrivals and resource states averages out, and each arrival effectively sees a deterministic fraction of available resources of each type. At the same time, the per-resource interaction structure of the original problem is preserved.

The (m, n) -scaling admits a natural operational interpretation as a large-scale, distributed matching system with limited visibility, depicted in Figure 3.



(a) Original system

(b) n -scaled system(c) m -scaled system

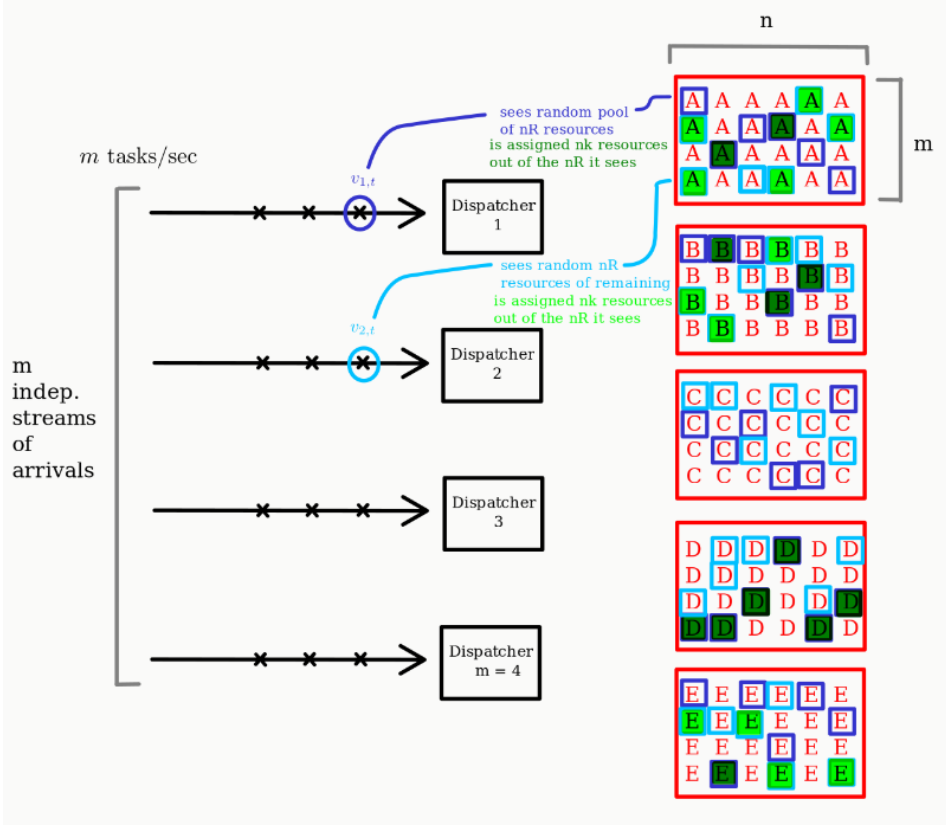


Fig. 3. (m, n) -scaled system ($m=4, n=6, R=5, k=2$): m independent streams share $mn=24$ copies of each resource type (A–E), split into m groups of n each per timestep. Blue cell = Dispatcher₁'s partition ($nR=30$ total); green cell = assigned this step ($nk=12$ total)

- (1) First, the m arrivals at each time step can be viewed as m independent *dispatchers* (or frontends), each receiving one task. Rather than a single centralized decision-maker with access to the entire resource pool, decisions are made locally and in parallel.
- (2) Second, the random partitioning of the mn resources into m groups of size n models *limited access to resources*. Each dispatcher only observes a subset of size n of each resource type, rather than the full system of size mn . This reflects practical constraints in large systems, where querying or coordinating across all resources is infeasible due to latency or computational overhead. The partitioning can be interpreted as randomized load balancing or hashing of resources to dispatchers, refreshed at each time step.
- (3) Third, each resource is queried by exactly one dispatcher per time step. This enforces that a resource is matched based only on a *single* arriving context, as in the original problem. In particular, resources cannot compare across multiple simultaneous arrivals, avoiding the “buffet” effect of m -scaling where resources effectively choose their most preferred task from a batch.

We define (m, n) -scaling formally below.

DEFINITION 4.1 ((m, n)-SCALED SYSTEM). Fix $m, n \in \mathbb{N}$. The (m, n) -scaled system is defined as follows.

Resources. For each type $r \in [R]$, there are mn resources of type r .

Arrivals. For each timestep t , m i.i.d. tasks arrive

Distribution. The mn resources of each type are split into m uniform partitions, such that every resource sees a uniformly chosen arrival and each arrival sees n uniformly chosen resources of each type.

Assignment. Every arrival is assigned nk resources out of the nR distributed to it, or as many as are available.

For any policy π , let $\text{Reward}_{m,n}(\pi)$ denote the long-run average total reward per timestep under π in the (m, n) -scaled system. The LAGindex assigns each arriving task $v_{j,t}$ for $j \in [m]$ to the nk available resources with the largest index $L(v_{j,t}, r)$ out of the nR resources distributed to it.

In the remainder of the section we show LAG is asymptotically optimal in the (m, n) -scaled loss system: Its normalized reward converges to $\text{OPT}(\text{OCC-LP}(0))$ (achievability), and no work-conserving policy exceeds this value in the limit (upper bound). The upper bound combines Proposition 3.2, which bounds any policy by $\text{OPT}(\text{OCC-LP}(B_{m,n}))$, with the fact that blocking vanishes (Proposition 4.7), so the bound tends to $\text{OPT}(\text{OCC-LP}(0))$.

THEOREM 4.1 (ASYMPTOTIC OPTIMALITY OF LAG). Under Assumptions 4.2 and 4.1, in the (m, n) -scaled loss system,

$$\frac{\text{Reward}_{m,n}(\text{LAG})}{mn} \rightarrow \text{OPT}(\text{OCC-LP}(0)) \quad \text{as } m, n \rightarrow \infty,$$

and $\limsup_{m,n \rightarrow \infty} \frac{\text{Reward}_{m,n}(\pi)}{mn} \leq \text{OPT}(\text{OCC-LP}(0))$ for every work-conserving policy π . Hence LAG is asymptotically optimal.

Below we state the lemmas we use to prove Theorem 4.1. Proofs are deferred to Appendix B.

4.2 Lemmas Concerning the Waterfill Function

The proof of Theorem 4.1 relies on two properties of the OCC-LP optimum: its waterfill structure and the Lipschitz continuity of the waterfill map.

DEFINITION 4.2 (WATERFILLING). Let $b \geq 0$ be a budget, $c \in [0, 1]^R$ a vector of capacity fractions, and π a total order on $[R]$. An allocation $u \in \mathbb{R}^R$ is a waterfill of (b, c, π) if:

- (1) No allocation exceeds capacity: $0 \leq u_r \leq c_r$ for all $r \in [R]$.
- (2) The budget is fulfilled: $\sum_r u_r = b$; or all capacities are used: $u = c$.
- (3) Resources are filled in priority order: for all $r, r' \in [R]$, if $\pi(r) < \pi(r')$ and $u_r < c_r$, then $u_{r'} = 0$.

We write $u = \text{waterfill}(b, c, \pi)$ and $u_r = \text{waterfill}_r(b, c, \pi)$.

That is, $\text{waterfill}(b, c, \pi)$ fills capacities $c_{\pi(1)}, c_{\pi(2)}, \dots$ greedily in priority order until the budget b is exhausted or all capacities are used.

LEMMA 4.2 (2-LIPSCHITZ). For any fixed b and order π , the map $f: [0, 1]^R \rightarrow [0, 1]^R$ defined by $x \mapsto \text{waterfill}(b, x, \pi)$ is 2-Lipschitz with respect to the ℓ_1 -norm.

LEMMA 4.3 (WATERFILL STRUCTURE OF LP OPTIMUM). There exists an OCC-LP optimum (u^*, ϕ^*) such that for each $v \in [V]$,

$$u_{vr}^* = \lambda_v \cdot \text{waterfill}_r(k, \phi^*, L(v, \cdot)).$$

That is, $(u_{vr}^*)_{r \in [R]}$ is a waterfill with budget k , capacities $(\phi_r^*)_{r \in [R]}$, and order $r \mapsto L(v, r)$, scaled by λ_v .

4.3 The Fluid Differential Equations

Let $\mathcal{X} = \{x \in [0, 1]^{R \times (d+1)} : \sum_{i=0}^d x_{r,i} = 1 \ \forall r \in [R]\}$ be the fluid state space, where $x_{r,i}$ denotes the fraction of type- r resources with exactly i steps of service remaining ($i = 0$ means free).

DEFINITION 4.3. For $x \in \mathcal{X}$, the fluid assignment rate of LAG is

$$u_{vr}^{LAG}(x) = \lambda_v \cdot \text{waterfill}_r(k, (x_{r,0})_{r \in [R]}, L(v, \cdot)).$$

DEFINITION 4.4 (FLUID MAP). Define the map $G: \mathcal{X} \rightarrow \mathcal{X}$ by

$$\begin{aligned} G_{r,0}(x) &= x_{r,0} - \sum_v u_{vr}^{LAG}(x) + x_{r,1}, \\ G_{r,i}(x) &= x_{r,i+1} + \sum_v \Pr(D_v = i) \cdot u_{vr}^{LAG}(x), \quad i = 1, \dots, d-1, \\ G_{r,d}(x) &= \sum_v \Pr(D_v = d) \cdot u_{vr}^{LAG}(x). \end{aligned}$$

LEMMA 4.4. Let $G^{m,n}$ denote the (random) one-step transition map of the (m, n) -scaled system under LAG. For every $x \in \mathcal{X}^{m,n}$, $G^{m,n}(x)$ converges in probability to $G(x)$ as $m, n \rightarrow \infty$. Further, for every bounded continuous $f: [0, 1]^{R \times (d+1)} \rightarrow \mathbb{R}$,

$$\lim_{m, n \rightarrow \infty} \sup_{x \in \mathcal{X}^{m,n}} |\mathbb{E}[f(G^{m,n}(x))] - f(G(x))| = 0.$$

For any fixed m, n , let $\mu^{m,n}$ denote a stationary distribution of the (m, n) -scaled chain under LAG; at least one exists since $\mathcal{X}^{m,n}$ is finite.

LEMMA 4.5. For every m, n and every initial state $x(0) \in \mathcal{X}^{m,n}$, there exists a stationary distribution $\mu^{m,n} \in \mathbb{P}(\mathcal{X}^{m,n})$ of the (m, n) -scaled system under LAG such that

$$\frac{\text{Reward}_{m,n}(\text{LAG})}{mn} = \sum_{v,r} s_{vr} \int u_{vr}^{LAG}(x) d\mu^{m,n}(x) + O\left(\frac{1}{\sqrt{n}}\right).$$

4.4 Congestion vanishes

The upper-bound half of Theorem 4.1 needs the blocking probability to vanish. We show that in the mean-field limit an arriving task almost never finds fewer than k servers free. Let $X_{r,0}^{(j)}$ denote the fraction of type- r servers that dispatcher j sees free, and $x_{r,0}$ the fraction free in the whole system. A task at dispatcher j is short of servers exactly when $\sum_r X_{r,0}^{(j)} < k$.

Fix any work-conserving policy π , and let $\mu_\pi^{m,n}$ be a stationary distribution of the (m, n) -scaled chain under π . We argue in two steps: First that the whole system keeps plenty of servers free, then that each dispatcher's sample is accurate enough to inherit this. The first step writes $\gamma = R(1 - \rho) - k$ for the number of free servers in excess of the largest task; Assumption 4.1 makes $\gamma > 0$. The total number of free servers stays above k with probability tending to one.

LEMMA 4.6. Let π be any work-conserving policy with stationary distribution $\mu_\pi^{m,n}$. Under Assumption 4.1, with $\gamma = R(1 - \rho) - k > 0$,

$$\Pr_{X \sim \mu_\pi^{m,n}} \left[\sum_r x_{r,0} \leq k + \frac{\gamma}{2} \right] \rightarrow 0 \quad \text{as } m, n \rightarrow \infty.$$

The second step accounts for the fact that a dispatcher sees only a sample of n servers per type, not the whole system. The next proposition shows the sample is accurate enough that a task almost never finds itself short.

PROPOSITION 4.7. *Under Assumption 4.1, for any work-conserving policy the stationary probability that an arriving task finds fewer than k free servers among those it observes, $P_{m,n}^{\text{block}}$, tends to 0 as $m, n \rightarrow \infty$.*

4.5 Proof of Optimality

ASSUMPTION 4.2 (GLOBAL ATTRACTOR PROPERTY). *There exists a unique $x^* \in \mathcal{X}$ such that for every initial $x(0) \in \mathcal{X}$,*

$$G^t(x(0)) \rightarrow x^* \quad \text{as } t \rightarrow \infty.$$

As noted in [25], this global attractor property was imposed in [27] to establish the asymptotic optimality of Whittle's index policy in a setting with a fixed population of symmetric bandits. In practice, this property is typically difficult to verify analytically and is often established through numerical evidence.

LEMMA 4.8. *Given Assumption 4.2, for any sequence $((m_k, n_k))_{k=1}^\infty$ with $m_k, n_k \rightarrow \infty$, the stationary distributions μ^{m_k, n_k} converge weakly to δ_{x^*} .*

PROOF SKETCH OF THEOREM 4.1. We sketch both halves; full details are in Appendix B.

Achievability. By Lemma 4.5, the normalized reward of LAG equals $\sum_{v,r} s_{vr} \int u_{vr}^{\text{LAG}}(x) d\mu^{m,n}(x)$ up to an $O(1/\sqrt{n})$ error, so it is governed by the stationary law $\mu^{m,n}$. Assumption 4.2 and Lemma 4.8 pin this law down: $\mu^{m,n}$ concentrates on the unique fluid fixed point x^* , so the reward converges to $\sum_{v,r} s_{vr} u_{vr}^{\text{LAG}}(x^*)$. It remains to identify x^* . Assembling a fluid state from the waterfill-structured OCC-LP optimum of Lemma 4.3 yields a fixed point of the fluid map G (Definition 4.4); uniqueness (Assumption 4.2) forces x^* to be this state, so $u^{\text{LAG}}(x^*) = u^*$ and the reward converges to $\text{OPT}(\text{OCC-LP}(0))$.

Upper bound. For any work-conserving policy π , Proposition 3.2 bounds its normalized reward by $\text{OPT}(\text{OCC-LP}(B_{m,n}))$, where $B_{m,n}$ is the blocking probability of the (m, n) -scaled system. Blocking is exactly the shortage event of Proposition 4.7, so $B_{m,n} = P_{m,n}^{\text{block}} \rightarrow 0$. The OCC-LP value is continuous in its demand at $B = 0$, since strict underload (Assumption 4.1) places the demand in the interior of the feasible region; hence $\text{OPT}(\text{OCC-LP}(B_{m,n})) \rightarrow \text{OPT}(\text{OCC-LP}(0))$. The two bounds meet, so LAG is asymptotically optimal. $\square$

5 Experiments

We evaluate LAG on seven instances spanning real peer-review datasets, synthetic call-center routing, and structured synthetic regimes, comparing against Greedy, Random, ALG-SC-LP [10], and Whittle [13] (Section 5.1). Our main findings are:

- LAG achieves 83–95% of OCC-UB across all instances at load $\rho = 0.7$, the highest reward among all online policies on every instance (Table 2).
- On real peer-review datasets, LAG outperforms Greedy by 6–10 percentage points at $\rho = 0.7$; on synthetic instances the gap reaches up to 23 percentage points (37% relative).
- In the mean-field limit, LAG converges to OCC-UB on all five peer-review and structured synthetic instances, confirming Theorem 4.1 empirically (Section 5.3).

5.1 Policies Evaluated

We compare the following policies and reference bounds throughout Section 5.

LAG (our policy). The Lagrangian index policy defined in Section 3.2. Upon arrival of a type- v task, LAG assigns the k_v available servers with the largest Lagrangian index values $L(v, r) = s_{vr} - \ell_v^* - \mathbb{E}[D_{v,r}] q_r^*$ (Definition 3.1). The dual variables (ℓ^*, q^*) are computed once by solving the OCC-LP offline; no further LP solves are needed at runtime.

Baselines.

Greedy: At each epoch, assigns the k_v available servers with the largest current scores s_{vnr} , ignoring future demand entirely. Variants of this approach are used in automated online assignment systems [12, 30].

Random: Assigns k_v servers chosen uniformly at random from all currently available servers, ignoring scores entirely. Serves as a lower-bound sanity check.

ALG-SC-LP [10]: The safe-choice LP policy of [10]. It solves the KIID LP offline for a fractional allocation x_{vr}^* , and at each epoch selects server r with probability proportional to x_{vr}^* among available servers, defaulting to Greedy when no available server has positive LP weight.

Whittle [13]: A closed-form index policy derived via the subsidy-of-indifference approach for restless multi-armed bandits [28]. The index is

$$W(v, r) = s_{vr} - \mathbb{E}[D_{v,r}] \sum_{v'} p_{v'} \max(s_{v'r} - s_{vr}, 0).$$

This index has a simple closed-form but provides no asymptotic optimality guarantee.

Upper Bounds. The following LP optimal values are not online policies; they are shown as reference lines in all figures.

OCC-UB: Optimal value of the OCC-LP (Section 3.1), an upper bound on the expected reward of any work-conserving policy (Proposition 3.2). At finite scale we plot $\text{OPT}(\text{OCC-LP}, B=\hat{B})$, evaluated at the empirical blocking rate $\hat{B}$ measured during simulation.

KIID-UB: Optimal value of the KIID LP of [10]; it equals OCC-UB when $k_v = 1$ and is strictly looser otherwise.

5.2 How Does Performance vary with Load

We measure reward as a function of system load $\rho = \lambda k \mathbb{E}[D]/R$, where $\lambda = \sum_v \lambda_v$ is the total arrival rate. We sweep ρ by varying λ while holding all other parameters fixed. At each load point we solve the OCC-LP once and compute LAG indices from the resulting dual variables; all other policies use only the score matrix and, where applicable, their respective LPs. We test six instances across three domains: two from real peer-review data, two synthetic call-center instances, and two analytical instances. Table 2 summarizes results at $\rho = 0.7$; the full load sweep appears in Table 3 in the appendix.

Table 2. Reward as % of OCC-UB at load $\rho = 0.7$.

Instance	LAG	Whittle	ALG-SC-LP	Greedy	Random
TMLR	87.8	83.2	80.5	78.4	49.6
ICLR	94.5	92.5	90.0	88.9	72.0
Specialist	90.6	85.5	84.6	85.5	55.0
Priority	85.6	85.6	82.2	63.0	54.9
Unfriendly	88.7	88.7	82.2	66.8	42.7
Low-rank	94.3	71.9	91.8	86.3	67.1

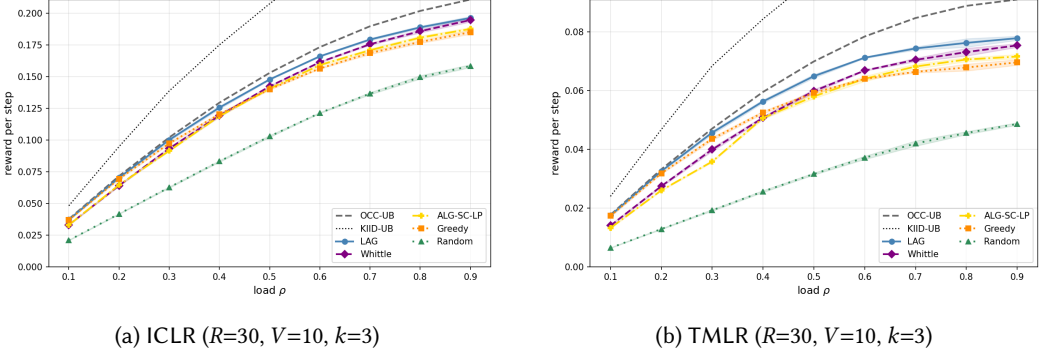


Fig. 4. Reward per step vs. load on real peer-review datasets. LAG tracks the OCC-UB across all loads.

In Figure 4, arriving tasks represent papers and servers represent reviewers ($k = 3$). Match scores are tf-idf cosine similarities [18] normalized to $[0, 1]$; service times are drawn from the empirical TMLR review-duration distribution [13]. We subsample $R = 30$ reviewers and $V = 10$ papers uniformly at random from each full matrix. For Figure 4a we use the ICLR 2018 similarity score matrix [29]; for Figure 4b we use the TMLR journal similarity matrix [13]. On TMLR, LAG reaches 87.8% of OCC-UB at $\rho = 0.7$ while Greedy falls to 78.4%, a gap that widens at high load. On ICLR the gap is smaller (94.5% vs. 88.9%) but persistent across all loads.

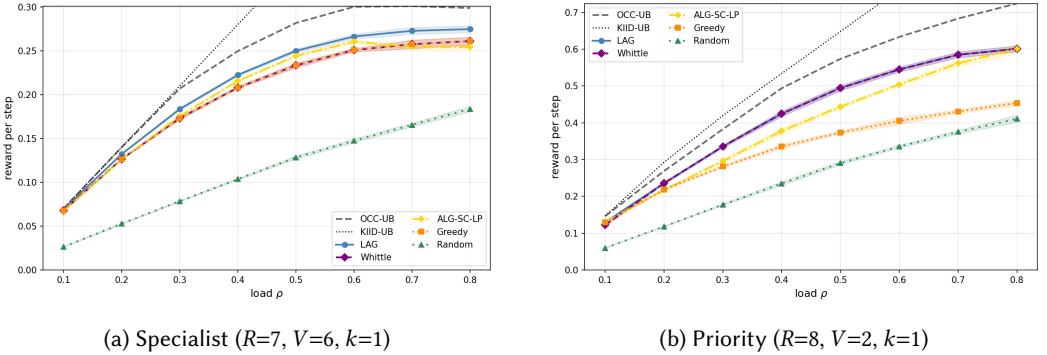


Fig. 5. Reward per step vs. load on synthetic call-center instances ($K=1$, geometric service).

Figure 5 presents two synthetic call-center instances with $K = 1$ and geometric service times.

The **Specialist** instance ($R = 7$, $V = 6$, $\mathbb{E}[D] = 10$) has two generalists that score 1.0 on all six call types and five specialists that score 1.0 only on their native routine type. A hard type with no specialist arrives three times as often as each routine type. For a routine call, both generalists and the native specialist score 1.0; Greedy breaks the three-way tie at random, wasting generalist capacity that only generalists can provide to hard calls. At $\rho = 0.7$, Greedy and Whittle both reach 85.5% of OCC-UB; LAG attains 90.6% by routing routine calls to native specialists via the per-type dual prices ℓ_v^* .

The **Priority** instance ($R = 8$, $V = 2$, $\mathbb{E}[D] = 20$) has two priority handlers scoring $[1.8, 8.0]^\top$ on (standard, priority) calls and six standard handlers scoring $[1.0, 0.2]^\top$. Standard calls arrive 70% of the time. Because priority handlers outscore standard handlers on standard calls, Greedy assigns

them first to standard arrivals, blocking capacity for high-reward priority calls. At $\rho = 0.7$, LAG and Whittle both achieve 85.6% of OCC-UB while Greedy falls to 63.0%.

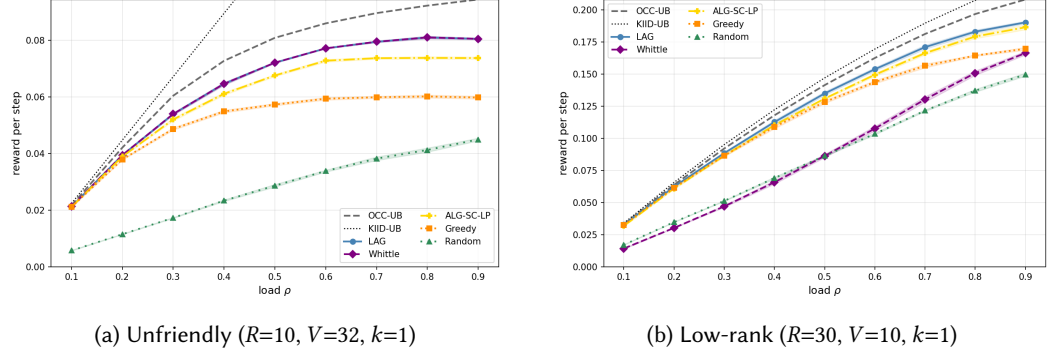


Fig. 6. Reward per step vs. load on synthetic instances. On Unfriendly, Greedy wastes high-quality servers on poor-match tasks. On Low-rank, Whittle falls more than 22 percentage points below LAG.

Figure 6 presents two synthetic instances designed to expose complementary failure modes. Both use deterministic service ($D = \mathbb{E}[D] = 43$ time steps) and $k = 1$.

The **Unfriendly** instance ($R = 10, V = 32$) has $g = 5$ good server types and $b = 5$ dummy server types. Good server $r \in [g]$ scores $s_{or} = 1.0$ for the subset of paper types it is well-matched to (those whose binary representation has bit r set) and $s_{or} = 0.01$ otherwise; dummy servers score 0 on all paper types. The $V = 2^g = 32$ task types cover all binary patterns of the g good servers, arriving uniformly. Because the near-zero score of 0.01 is strictly positive, Greedy assigns a good server even when the match is poor, burning capacity rather than absorbing the arrival with a dummy server and holding the good server for a future high-reward match. At $\rho = 0.7$, Greedy achieves only 66.8% of OCC-UB while LAG attains 88.7%, a relative improvement of 33%. LAG and Whittle coincide on this instance ($k = 1$): Both correctly identify that the value of withholding a well-matched server outweighs the immediate reward of a poor match.

The **Low-rank** instance ($R = 30, V = 10$) uses a rank-1 score matrix $s_{or} = a_o b_r$, where $a_o = v/(V-1)$ and $b_r = r/(R-1)$ are linearly spaced in $[0, 1]$, and task types arrive uniformly. LAG performs best (94.3% of OCC-UB at $\rho = 0.7$), followed by ALG-SC-LP (91.8%) and Greedy (86.3%). Whittle falls to 71.9%, more than 22 percentage points below LAG and more than 14 points below Greedy.

5.3 Convergence to the Mean-Field Limit

We test Theorem 4.1 empirically. Using the same five instances at load $\rho = 0.8$, we scale the system under the (m, n) -scaling with $m = \lfloor \sqrt{n} \rfloor$, sweeping $n \in \{1, 4, 16, 64, 256, 1024, 4096, 16384, 65,536\}$. Simulation length scales adaptively as $T \propto 1/m$ so that total computational work stays roughly constant across n values. Each data point is the mean of 3 independent trials; shaded bands show $\pm 1.96 \hat{\sigma} / \sqrt{3}$ confidence intervals.

Figure 7 plots LAG's reward per server copy as a percentage of OCC-UB for all five instances on a log n axis. All instances converge toward 100% of OCC-UB as n grows. At $n = 65,536$ ($m = 256$), LAG reaches 99.1% on TMLR, 99.6% on ICLR, 99.9% on Specialist, 99.8% on Unfriendly, and 100.0% on Low-rank. Convergence is fastest on Low-rank, which crosses 99% at $n = 256$; it is slowest on Unfriendly, where LAG starts at 86.8% at $n = 1$ but climbs steadily to 99.8% by $n = 65,536$.

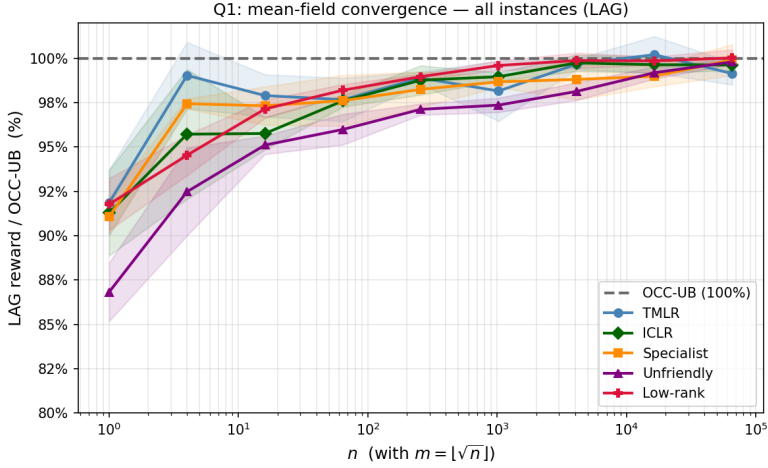


Fig. 7. LAG’s reward per server copy, $\text{reward}/(mn)$, as a percentage of OCC-UB vs. n under the (m, n) -scaling ($m = \lfloor \sqrt{n} \rfloor$). All five instances converge to OCC-UB as n grows, confirming Theorem 4.1.

6 Conclusion

We studied online assignment of reusable, quality-differentiated resources, where each arriving task must be matched immediately to k servers that then become busy for a random duration. We cast the problem as a contextual restless multi-armed bandit and derived the occupancy-measure LP, whose optimum upper-bounds the reward of any work-conserving policy. The dual of this LP decouples the routing problem across servers and yields LAG, a closed-form index policy: Each task-type and server pair is scored by its raw match value less an opportunity-cost term that reflects the value of keeping that server free for future arrivals. To analyze LAG we introduced an (m, n) -scaling regime, in which the number of copies of each server and the number of arrivals per step grow together; scaling the arrivals concentrates the context distribution and restores the deterministic fluid limit that a contextual bandit otherwise lacks. In this regime we proved LAG is asymptotically optimal, attaining the LP bound. Empirically, on real peer-review traces and synthetic instances, LAG achieves 94–99% of the LP upper bound and improves over the natural greedy policy by up to 20% on peer-review data, including well outside the regime where our optimality theory applies.

References

- [1] Ivo Adan and Gideon Weiss. 2014. A Skill Based Parallel Service System Under FCFS-ALIS — Steady State, Overloads, and Abandonments. *Stochastic Systems* 4, 1 (2014), 250–299. doi:10.1214/13-SSY117
- [2] Sigrún Andradóttir, Hayriye Ayhan, and Douglas G. Down. 2003. Dynamic Server Allocation for Queueing Networks with Flexible Servers. *Operations Research* 51, 6 (2003), 952–968. doi:10.1287/opre.51.6.952.24921
- [3] Mor Armony. 2005. Dynamic Routing in Large-Scale Service Systems with Heterogeneous Servers. *Queueing Systems* 51 (2005), 287–329. doi:10.1007/s11134-005-3760-7
- [4] Rami Atar. 2005. Scheduling Control for Queueing Systems with Many Servers: Asymptotic Optimality in Heavy Traffic. *The Annals of Applied Probability* 15, 4 (2005), 2606–2650. doi:10.1214/105051605000000601
- [5] Konstantin Avrachenkov, Vivek S Borkar, and Pratik Shah. 2026. Lagrangian index policy for restless bandits with average reward. *Queueing Systems* 110, 1 (2026), 21.
- [6] Michel Benaïm and Jean-Yves Le Boudec. 2008. A class of mean field interaction models for computer and communication systems. *Performance evaluation* 65, 11-12 (2008), 823–838.
- [7] Patrick Billingsley. 2013. *Convergence of probability measures*. John Wiley & Sons.

- [8] Xin Chen and I-Hong Hou. 2024. Contextual restless multi-armed bandits with application to demand response decision-making. In *2024 IEEE 63rd Conference on Decision and Control (CDC)*. 2652–2657.
- [9] Steven Delong, Alireza Farhadi, Rad Niazadeh, Balasubramanian Sivan, and Rajan Udawani. 2024. Online Bipartite Matching with Reusable Resources. *Mathematics of Operations Research* 49, 3 (2024), 1825–1854. doi:10.1287/moor.2022.0242 Published online: October 13, 2023.
- [10] John P. Dickerson, Karthik A. Sankararaman, Aravind Srinivasan, and Pan Xu. 2021. Allocation Problems in Ride-sharing Platforms: Online Matching with Offline Reusable Resources. *ACM Transactions on Economics and Computation* 9, 3 (September 2021), 1–17. doi:10.1145/3456756
- [11] Rick Durrett. 2019. *Probability: theory and examples*. Vol. 49. Cambridge university press.
- [12] Hui Fang and ChengXiang Zhai. 2007. Probabilistic Models for Expert Finding. In *Advances in Information Retrieval*, Giambattista Amati, Claudio Carpineto, and Giovanni Romano (Eds.). Springer, 418–430.
- [13] Keerthana Gurushankar, Nihar Shah, Alan Scheller-Wolf, and Mor Harchol-Balter. 2026. The Online Reviewer Assignment Problem. Technical report.
- [14] Zhiyi Huang, Zhihao Gavin Tang, and David Wajc. 2024. Online Matching: A Brief Survey. *ACM SIGecom Exchanges* 22, 1 (October 2024), 135–158. doi:10.1145/3699824.3699837
- [15] Frank P. Kelly. 1991. Loss networks. *The Annals of Applied Probability* 1, 3 (1991), 319–378.
- [16] Maialen Larrañaga. 2015. *Dynamic control of stochastic and fluid resource-sharing systems*. Ph.D. Dissertation. Institut National Polytechnique de Toulouse-INPT; Universidad del País Vasco.
- [17] Avishai Mandelbaum and Alexander L. Stolyar. 2004. Scheduling Flexible Servers with Convex Delay Costs: Heavy-Traffic Optimality of the Generalized $c\mu$ -Rule. *Operations Research* 52, 6 (2004), 836–855. doi:10.1287/opre.1040.0152
- [18] Christopher D Manning. 2008. *Introduction to information retrieval*. Cambridge University Press.
- [19] José Niño-Mora. 2023. Markovian restless bandits and index policies: A review. *Mathematics* 11, 7 (2023), 1–27.
- [20] Christos H Papadimitriou and John N Tsitsiklis. 1987. The complexity of Markov Decision Processes. *Mathematics of Operations Research* 12, 3 (1987), 441–450.
- [21] Robert J Serfling. 1974. Probability inequalities for the sum in sampling without replacement. *The Annals of Statistics* (1974), 39–48.
- [22] M. Shanks, G. Yu, and S. H. Jacobson. 2023. Approximation Algorithms for Stochastic Online Matching with Reusable Resources. *Mathematical Methods of Operations Research* 98 (2023), 43–56. doi:10.1007/s00186-023-00822-3
- [23] Kyle Siegrist. 2022. The Multivariate Hypergeometric Distribution. Section 12.3. License: CC BY 2.0. Accessed: April 2026.
- [24] Hanna Sumita, Shinji Ito, Kei Takemura, Daisuke Hatano, Takuro Fukunaga, Naonori Kakimura, and Ken-ichi Kawarabayashi. 2022. Online Task Assignment Problems with Reusable Resources. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 5199–5207. doi:10.1609/aaai.v36i5.20455
- [25] I. M. Verloop. 2016. ASYMPTOTICALLY OPTIMAL PRIORITY POLICIES FOR INDEXABLE AND NONINDEXABLE RESTLESS BANDITS. *The Annals of Applied Probability* 26, 4 (2016), 1947–1995. <http://www.jstor.org/stable/24810047>
- [26] Rodney B. Wallace and Ward Whitt. 2005. A Staffing Algorithm for Call Centers with Skill-Based Routing. *Manufacturing & Service Operations Management* 7, 4 (2005), 276–294. doi:10.1287/msom.1050.0086
- [27] Richard R Weber and Gideon Weiss. 1990. On an index policy for restless bandits. *Journal of Applied Probability* 27, 3 (1990), 637–648.
- [28] Peter Whittle. 1988. Restless Bandits: Activity Allocation in a Changing World. *Journal of Applied Probability* 25, A (1988), 287–298. doi:10.2307/3214163
- [29] Yichong Xu, Han Zhao, Xiaofei Shi, and Nihar B Shah. 2019. On strategyproof conference peer review. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence*. 616–622.
- [30] Xiquan Zhao and Yangsen Zhang. 2022. Reviewer assignment algorithms for peer review automation: A survey. *Information Processing & Management* 59, 5 (sep 2022), 20 pages. doi:10.1016/j.ipm.2022.103028

A Deferred Proofs for the Derivation of LAG

PROOF OF LEMMA 3.4. For any Arm- r bandit policy π , let u_{or}^π be the long-run rate of type- v assignments to server r , and let ϕ_r^π be the long-run free fraction of server r . By the argument of Proposition 3.2 applied to a single server (Little’s Law for the capacity constraint, and the martingale bound for the availability constraint), (u^π, ϕ^π) is feasible for the Arm- r LP. Thus the average reward of π is $g_\pi = \sum_v (s_{or} - \ell_v) u_{or}^\pi \leq q_r^*$.

To show the bound is tight, given the Arm- r LP optimal solution (u^*, ϕ^*) , construct bandit policy $\hat{\pi}$ that, when free and a type- v task arrives, accepts with probability $u_{vr}^*/(\lambda_v \phi_r^*)$. The long-run assignment rate under $\hat{\pi}$ satisfies:

$$u_{vr}^{\hat{\pi}} = \frac{u_{vr}^*}{\lambda_v \phi_r^*} \cdot \lambda_v \phi_r^{\hat{\pi}} = \frac{u_{vr}^*}{\phi_r^*} \cdot \phi_r^{\hat{\pi}}.$$

Applying Little's Law: $\phi_r^{\hat{\pi}} = 1 - \sum_v \mathbb{E}[D_v] u_{vr}^{\hat{\pi}} = 1 - \frac{\phi_r^{\hat{\pi}}}{\phi_r^*} \sum_v \mathbb{E}[D_v] u_{vr}^* = 1 - \frac{\phi_r^{\hat{\pi}}}{\phi_r^*} (1 - \phi_r^*)$. Solving gives $\phi_r^{\hat{\pi}} = \phi_r^*$, hence $u_{vr}^{\hat{\pi}} = u_{vr}^*$, and the average reward of $\hat{\pi}$ equals $\sum_v (s_{vr} - \ell_v) u_{vr}^* = q_r^*$. $\square$

PROOF OF LEMMA 3.5. We use the average-reward Bellman optimality equation. Let g be the optimal average reward per step and $h(\cdot)$ a relative value function, normalised so that $h(0) = 0$ at the free state. A busy server with $x \geq 1$ steps remaining takes no action and earns no reward, advancing deterministically to state $x-1$, so $g+h(x) = h(x-1)$; with $h(0) = 0$ this gives $h(x) = -gx$.

When the server is free and a type- v task arrives, accepting earns $s_{vr} - \ell_v$ and moves the server to state D_v , while rejecting earns 0 and keeps it free. Accepting is optimal iff its value is at least that of rejecting:

$$(s_{vr} - \ell_v) + \mathbb{E}[h(D_v)] \geq h(0), \quad \text{i.e.} \quad s_{vr} - \ell_v \geq g \mathbb{E}[D_v],$$

since $\mathbb{E}[h(D_v)] = -g \mathbb{E}[D_v]$. By Lemma 3.4, $g = q_r^*$, giving the stated threshold. $\square$

B Deferred Proofs for Mean Field Optimality

PROOF OF LEMMA 4.2. Without loss of generality let π be the order $1 > 2 > \dots > R$. We start by considering a single-coordinate perturbation. Let $x' = x + \delta e_j$ for $\delta > 0$, where e_j is the j -th standard basis vector. Increasing the capacity of resource j by δ leaves all higher-priority allocations $f_i(x)$, $i < j$, unchanged. The allocation $f_j(x)$ increases by at most δ . Allocations to lower-priority resources can only decrease, and the total decrease is at most the increase at j (since total allocation is capped at b). Hence:

$$\begin{aligned} \|f(x') - f(x)\|_1 &= (f_j(x') - f_j(x)) - \sum_{i>j} (f_i(x') - f_i(x)) \\ &\leq 2(f_j(x') - f_j(x)) \leq 2\delta. \end{aligned}$$

A symmetric argument applies for $\delta < 0$. For a general perturbation $x' - x = \delta \in [-1, 1]^R$, perturbing one coordinate at a time and applying the triangle inequality gives:

$$\begin{aligned} \|f(x') - f(x)\|_1 &\leq \sum_{j=1}^R \left\| f\left(x + \sum_{i=1}^j \delta_i e_i\right) - f\left(x + \sum_{i=1}^{j-1} \delta_i e_i\right) \right\|_1 \\ &\leq \sum_{j=1}^R 2|\delta_j| = 2\|x' - x\|_1. \end{aligned} \quad \square$$

PROOF OF LEMMA 4.3. By Lemma 3.5, the optimal Arm- r bandit policy accepts type v when free if and only if $L(v, r) \geq 0$. By Lemma 3.4, the occupancy measure (u^*, ϕ^*) of this policy achieves the Arm- r LP optimum, so Proposition 3.3 gives:

$$\begin{aligned} L(v, r) > 0 &\implies u_{vr}^* = \lambda_v \phi_r^*, \\ L(v, r) = 0 &\implies u_{vr}^* \in [0, \lambda_v \phi_r^*], \\ L(v, r) < 0 &\implies u_{vr}^* = 0. \end{aligned}$$

The demand constraint (2) of OCC-LP(0) is the equality $\sum_r u_{vr}^* = k\lambda_v$, and the dual multiplier $\ell_v^* \in \mathbb{R}$ is the level at which it binds: Since $L(v, r) = s_{vr} - \ell_v^* - \mathbb{E}[D_v]$ q_r^* shifts every index by the same amount as ℓ_v^* varies, ℓ_v^* is chosen so that filling the positive-index capacities exactly meets the budget. By Proposition 3.3, $u_{vr}^* = \lambda_v \phi_r^*$ for all r with $L(v, r) > 0$ and $u_{vr}^* = 0$ for all r with $L(v, r) < 0$. The residual budget

$$\beta_v := k\lambda_v - \sum_{r: L(v, r) > 0} \lambda_v \phi_r^*$$

is then exactly the mass the optimum places on the equal-index resources: Using primal feasibility of the demand constraint (2), $\sum_r u_{vr}^* = k\lambda_v$, so

$$\beta_v = \sum_r u_{vr}^* - \sum_{r: L(v, r) > 0} u_{vr}^* = \sum_{r: L(v, r) = 0} u_{vr}^*.$$

Hence $0 \leq \beta_v \leq \sum_{r: L(v, r) = 0} \lambda_v \phi_r^*$, the first inequality by nonnegativity and the second by the bound $u_{vr}^* \leq \lambda_v \phi_r^*$ of (4). The equal-index resources therefore hold exactly enough capacity to absorb β_v ; we distribute it across them in any order. The resulting $(u_{vr}^*)_r$ satisfies all three conditions of Definition 4.2 with budget $k\lambda_v$, capacities $(\lambda_v \phi_r^*)_r$, and order $r \mapsto L(v, r)$. Dividing by λ_v gives $u_{vr}^*/\lambda_v = \text{waterfill}_r(k, \phi^*, L(v, \cdot))$. $\square$

PROOF OF LEMMA 4.4. Fix $x \in \mathcal{X}^{m,n}$. In the (m, n) -scaled system, the fraction $x_{r,i}$ is a multiple of $1/(mn)$, so $mn \cdot x_{r,i}$ is the count of type- r resources with i steps remaining. We first construct the random objects comprising one step of $G^{m,n}$, then bound $\|G^{m,n}(x) - G(x)\|_\infty$ by applying concentration inequalities to each source of randomness.

The mn resources of each type are split uniformly at random into m groups of n . Let $nX_{r,i}^{(j)}$ denote the count of type- r , level- i resources landing in partition j . This vector follows a multivariate hypergeometric distribution with population mn , stratum sizes $mn \cdot x_{r,i}$, and sample size n [23], so

$$\mathbb{E}[X_{r,i}^{(j)}] = x_{r,i}, \quad \text{Var}(X_{r,i}^{(j)}) = x_{r,i}(1 - x_{r,i}) \cdot \frac{m-1}{mn-1} \leq \frac{1}{n}. \quad (6)$$

Each partition j receives an independent arrival V_j , where $\Pr(V_j = v) = \lambda_v$ and $\Pr(V_j = \emptyset) = 1 - \sum_v \lambda_v$. Let $nU_{vr}^{(j)}$ denote the number of type- (v, r) assignments made by LAG in partition j . Since LAG applies the waterfill to the free resources it observes:

$$U_{vr}^{(j)} = \mathbf{1}[V_j = v] \cdot \text{waterfill}_r(k, (X_{r,0}^{(j)})_{r \in [R]}, L(v, \cdot)). \quad (7)$$

Each of the $nU_{vr}^{(j)}$ assigned resources independently draws a service time $D_v \in \{1, \dots, d\}$. Let $U_{vr,i}^{(j)}$ denote the fraction of partition- j type- (v, r) assignments with $D_v = i$, so $nU_{vr,i}^{(j)} \sim \text{Binomial}(nU_{vr}^{(j)}, \Pr(D_v = i))$.

After recombining the m partitions and aging all busy resources by one step:

$$\begin{aligned} G_{r,0}^{m,n}(x) &= x_{r,0} - \frac{1}{m} \sum_j \sum_v U_{vr}^{(j)} + x_{r,1}, \\ G_{r,i}^{m,n}(x) &= x_{r,i+1} + \frac{1}{m} \sum_j \sum_v U_{vr,i}^{(j)}, \quad i = 1, \dots, d-1, \\ G_{r,d}^{m,n}(x) &= \frac{1}{m} \sum_j \sum_v U_{vr,d}^{(j)}. \end{aligned}$$

It remains to bound $\|G^{m,n}(x) - G(x)\|_\infty$. By the triangle inequality, for $i \geq 1$:

$$\begin{aligned} |G_{r,i}^{m,n}(x) - G_{r,i}(x)| &\leq \underbrace{\left| \frac{1}{m} \sum_j \sum_v U_{vr,i}^{(j)} - \Pr(D_v = i) \cdot \frac{1}{m} \sum_j \sum_v U_{vr}^{(j)} \right|}_{\text{service drift}} \\ &\quad + \Pr(D_v = i) \cdot \left| \frac{1}{m} \sum_j \sum_v U_{vr}^{(j)} - \sum_v u_{vr}^{\text{LAG}}(x) \right|. \end{aligned} \quad (8)$$

The second factor also appears in the $i = 0$ component:

$$|G_{r,0}^{m,n}(x) - G_{r,0}(x)| = \left| \frac{1}{m} \sum_j \sum_v U_{vr}^{(j)} - \sum_v u_{vr}^{\text{LAG}}(x) \right|. \quad (9)$$

Write $\phi = (x_{r,0})_r$ and $\phi^{(j)} = (X_{r,0}^{(j)})_r$. By (7) and the triangle inequality:

$$\begin{aligned} \left| \frac{1}{m} \sum_j \sum_v U_{vr}^{(j)} - \sum_v u_{vr}^{\text{LAG}}(x) \right| &\leq \underbrace{\frac{1}{m} \sum_j \sum_v |\text{waterfill}_r(k, \phi^{(j)}, L(v, \cdot)) - \text{waterfill}_r(k, \phi, L(v, \cdot))|}_{\text{partition drift}} \\ &\quad + \underbrace{\left| \frac{1}{m} \sum_j \text{waterfill}_r(k, \phi, L(V_j, \cdot)) - \sum_v \lambda_v \text{waterfill}_r(k, \phi, L(v, \cdot)) \right|}_{\text{arrival drift}}, \end{aligned} \quad (10)$$

with the convention $\text{waterfill}_r(k, \phi, L(\emptyset, \cdot)) = 0$ when $V_j = \emptyset$ (no task arrives).

For the partition drift, by Lemma 4.2 (2-Lipschitz in ℓ_1):

$$\text{partition drift} \leq \frac{2}{m} \sum_j \sum_{r'} |\phi_{r'}^{(j)} - \phi_{r'}|.$$

By the Serfling inequality [21] for sampling without replacement:

$$\Pr(|\phi_{r'}^{(j)} - \phi_{r'}| > \delta) \leq 2e^{-n\delta^2}.$$

For the arrival drift, each term $\text{waterfill}_r(k, \phi, L(V_j, \cdot)) \in [0, 1]$ is i.i.d. with mean $\sum_v \lambda_v \text{waterfill}_r(k, \phi, L(v, \cdot))$.

By Hoeffding's inequality:

$$\Pr(\text{arrival drift} > \delta) \leq 2e^{-2m\delta^2}.$$

For the service drift, conditional on $U_{vr}^{(j)} = u$, $nU_{vr,i}^{(j)} \sim \text{Binomial}(nu, \Pr(D_v = i))$, so $U_{vr,i}^{(j)} - \Pr(D_v = i) \cdot U_{vr}^{(j)}$ has mean zero and variance at most $1/(4n)$. By Hoeffding's inequality applied conditionally:

$$\Pr(|U_{vr,i}^{(j)} - \Pr(D_v = i) \cdot U_{vr}^{(j)}| > \delta) \leq 2e^{-8n\delta^2}.$$

Averaging over $j = 1, \dots, m$ and a union bound over the VR pairs (v, r) gives $\Pr(\text{service drift} > \delta) \leq 2VR e^{-8n\delta^2}$.

Collecting (8)–(10) and the three drift bounds:

$$\sup_{x \in \mathcal{X}^{m,n}} \Pr[\|G^{m,n}(x) - G(x)\|_\infty > \delta] \leq 4R \cdot e^{-n\delta^2} + 2V \cdot e^{-2m\delta^2} + 2VR \cdot e^{-8n\delta^2}. \quad (11)$$

The right side vanishes as $m, n \rightarrow \infty$ for any fixed $\delta > 0$, proving convergence in probability.

For the uniform convergence of $\mathbb{E}[f(G^{m,n}(x))]$: since $[0, 1]^{R \times (d+1)}$ is compact and f is continuous, f is uniformly continuous. For any $\epsilon > 0$, choose δ such that $\|y - z\|_\infty \leq \delta \Rightarrow |f(y) - f(z)| < \epsilon/2$. Let A be the event $\|G^{m,n}(x) - G(x)\|_\infty \leq \delta$. Then:

$$|\mathbb{E}[f(G^{m,n}(x))] - f(G(x))| \leq \frac{\epsilon}{2} + \Pr(A^c) \cdot 2\|f\|_\infty.$$

By (11), $\Pr(A^c) \rightarrow 0$ as $m, n \rightarrow \infty$, so $\limsup \leq \epsilon/2$. Since ϵ is arbitrary, the limit is 0. $\square$

PROOF OF LEMMA 4.5. Consider a T -step sample path. Let $X(t)$ be the state at time t and $nU_{vr}^j(t)$ the number of type- (v, r) assignments by LAG in partition $j \in [m]$ at time t . Then:

$$\frac{\text{Reward}_{m,n}(\text{LAG})}{mn} = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \frac{1}{m} \sum_{j=1}^m \sum_{v,r} s_{vr} U_{vr}^j(t).$$

Given $X(t) = x$, the arrival V_j is independent of the partition draw $(X_{r,i}^{(j)})$, with $\Pr(V_j = v) = \lambda_v$. By (7):

$$\mathbb{E}[U_{vr}^j \mid X(t) = x] = \lambda_v \cdot \mathbb{E}[\text{waterfill}_r(k, (X_{r,0}^{(j)})_r, L(v, \cdot)) \mid X(t) = x].$$

All m partitions are identically distributed, so summing over j and (v, r) :

$$\mathbb{E}\left[\frac{\text{Reward}(t)}{mn} \mid X(t) = x\right] = \sum_{v,r} s_{vr} \lambda_v \mathbb{E}[\text{waterfill}_r(k, (X_{r,0}^{(1)})_r, L(v, \cdot)) \mid X(t) = x], \quad (12)$$

where by (6): $\mathbb{E}[X_{r,0}^{(1)}] = x_{r,0}$ and $\text{Var}(X_{r,0}^{(1)}) \leq 1/n$.

By Definition 4.3, Lemma 4.2 (2-Lipschitz), and Cauchy-Schwarz:

$$|\mathbb{E}[\text{waterfill}_r(k, (X_{r,0}^{(1)})_r, L(v, \cdot))] - \text{waterfill}_r(k, (x_{r,0})_r, L(v, \cdot))| \leq 2 \sum_{r' \in [R]} \sqrt{\text{Var}(X_{r',0}^{(1)})} \leq \frac{2R}{\sqrt{n}}.$$

Summing over (v, r) , applying $|s_{vr}| \leq s_{\max}$, and using $\sum_v \lambda_v \leq 1$:

$$\left| \mathbb{E}\left[\frac{\text{Reward}(t)}{mn} \mid X(t) = x\right] - \sum_{v,r} s_{vr} u_{vr}^{\text{LAG}}(x) \right| \leq \frac{2R^2 s_{\max}}{\sqrt{n}} =: \frac{C}{\sqrt{n}}.$$

Taking expectation and averaging over $t \in [T]$:

$$\left| \frac{1}{T} \sum_{t=1}^T \mathbb{E}\left[\frac{\text{Reward}(t)}{mn}\right] - \frac{1}{T} \sum_{t=1}^T \sum_{v,r} s_{vr} \mathbb{E}[u_{vr}^{\text{LAG}}(X(t))] \right| \leq \frac{C}{\sqrt{n}}.$$

Since $\mathcal{X}^{m,n}$ is finite, the Cesàro averages $\frac{1}{T} \sum_{t=1}^T P^t(x(0), \cdot)$ converge to a stationary distribution $\mu^{m,n}$ for every initial state $x(0)$ [11, Eq. (5.6.1)]. Taking $T \rightarrow \infty$ gives the stated bound. $\square$

PROOF OF LEMMA 4.6. Each in-service task occupies nk servers and there are mn servers of each type, so the busy fraction is $R - \sum_r x_{r,0} = \frac{nk}{mn} N = \frac{k}{m} N$, where N is the number of tasks in service. Since $R(1 - \rho) = k + \gamma$, the event $\sum_r x_{r,0} \leq k + \frac{\gamma}{2}$ is exactly $\frac{k}{m} N \geq \rho R + \frac{\gamma}{2}$.

Index arrival slots by (j, s) with dispatcher $j \in [m]$ and age $s \in \{0, \dots, d-1\}$. In the loss model a served task starts service at its arrival, so a task is in service in the current state iff it was accepted at some slot (j, s) and its duration exceeds its age, $D > s$. Dropping the acceptance indicator,

$$N \leq N_\infty := \sum_{j=1}^m \sum_{s=0}^{d-1} \mathbf{1}[\text{slot } (j, s) \text{ holds a task with } D > s].$$

The md indicators are independent (distinct slots are distinct arrivals, with independent types and durations), and slot (j, s) is active with probability $\sum_v \lambda_v \Pr[D_v > s]$. Hence

$$\mathbb{E}[N_\infty] = m \sum_{s=0}^{d-1} \sum_v \lambda_v \Pr[D_v > s] = m \sum_v \lambda_v \mathbb{E}[D_v], \quad \frac{k}{m} \mathbb{E}[N_\infty] = k \sum_v \lambda_v \mathbb{E}[D_v] = \rho R.$$

By Hoeffding's inequality over these md independent indicators in $\{0, 1\}$,

$$\Pr\left[\frac{k}{m} N \geq \rho R + \frac{\gamma}{2}\right] \leq \Pr\left[N_\infty - \mathbb{E}[N_\infty] \geq \frac{\gamma m}{2k}\right] \leq \exp\left(-\frac{\gamma^2 m}{2k^2 d}\right) \xrightarrow{m \rightarrow \infty} 0.$$

Arrivals and durations are exogenous, independent of the policy and of the state history, so this bound holds under the stationary law $\mu_\pi^{m,n}$ of any work-conserving policy π , giving the claim. $\square$

PROOF OF PROPOSITION 4.7. Fix a type v and let γ be as in Lemma 4.6. By that lemma the system's free capacity exceeds $k + \gamma/2$ with probability tending to one, so it suffices to bound the probability on that event. There the task is short only if its dispatcher's free capacity falls short of the system's by more than $\gamma/2$, i.e. $\sum_r |X_{r,0}^{(1)} - x_{r,0}| > \gamma/2$. Each $X_{r,0}^{(1)}$ is a sample of n servers drawn without replacement, so by the Serfling inequality [21] (as in Lemma 4.4) and a union bound this has probability at most $2R e^{-n(\gamma/2R)^2}$, uniformly in x . Both contributions vanish; averaging over v gives $P_{m,n}^{\text{block}} \rightarrow 0$. $\square$

PROOF OF LEMMA 4.8. We show for every bounded continuous f that $\int f d\mu^{m,n} \rightarrow f(x^*)$, following the approach of [6].

First, every subsequence of $\{\mu^{m,n}\}$ has a weakly convergent sub-subsequence. Each $\mu^{m,n}$ is supported on $\mathcal{X}^{m,n} \subseteq [0, 1]^{R \times (d+1)}$. Since $[0, 1]^{R \times (d+1)}$ is compact, by Prokhorov's theorem [7, Theorem 5.1] every subsequence of $\{\mu^{m,n}\}$ has a weakly convergent sub-subsequence.

Second, every subsequential limit is G -invariant. Let $\mu^{m_{k_j}, n_{k_j}} \Rightarrow \mu$. Stationarity of $\mu^{m_{k_j}, n_{k_j}}$ gives:

$$\int f d\mu^{m_{k_j}, n_{k_j}} = \int \mathbb{E}[f(G^{m_{k_j}, n_{k_j}}(x))] d\mu^{m_{k_j}, n_{k_j}}.$$

The LHS converges to $\int f d\mu$. By Lemma 4.4, $\sup_x |\mathbb{E}[f(G^{m,n}(x))] - f(G(x))| \rightarrow 0$; since f is bounded, the RHS converges to $\int f(G(x)) d\mu^{m_{k_j}, n_{k_j}}(x)$. Since $x \mapsto f(G(x))$ is bounded continuous (G is Lipschitz by Lemma 4.2) and $\mu^{m_{k_j}, n_{k_j}} \Rightarrow \mu$, this further converges to $\int f(G(x)) d\mu(x)$. Thus $\int f d\mu = \int f(G(x)) d\mu(x)$, i.e., μ is G -invariant.

Third, the unique G -invariant measure is δ_{x^*} . For any G -invariant probability measure μ , induction gives $\int f d\mu = \int f(G^t(x)) d\mu(x)$ for all $t \geq 1$. By Assumption 4.2, $G^t(x) \rightarrow x^*$ pointwise, so by dominated convergence:

$$\int f d\mu = \lim_{t \rightarrow \infty} \int f(G^t(x)) d\mu(x) = f(x^*).$$

Hence $\mu = \delta_{x^*}$. Since every subsequential limit equals δ_{x^*} , the full sequence converges weakly to δ_{x^*} . $\square$

PROOF OF THEOREM 4.1. By Lemma 4.5 and the continuity of $x \mapsto u_{vr}^{\text{LAG}}(x)$ (Lemma 4.2):

$$\frac{\text{Reward}_{m,n}(\text{LAG})}{mn} = \sum_{v,r} s_{vr} \int u_{vr}^{\text{LAG}}(x) d\mu^{m,n}(x) + O\left(\frac{1}{\sqrt{n}}\right) \xrightarrow{\mu^{m,n} \Rightarrow \delta_{x^*}} \sum_{v,r} s_{vr} u_{vr}^{\text{LAG}}(x^*),$$

where the weak convergence is by Lemma 4.8. It remains to show $u^{\text{LAG}}(x^*) = u^*$.

Let (u^*, ϕ^*) be the OCC-LP optimum from Lemma 4.3, and define $\hat{x} \in \mathcal{X}$ by

$$\hat{x}_{r,0} = \phi_r^*, \quad \hat{x}_{r,i} = \sum_v \Pr(D_v \geq i) u_{vr}^*, \quad i = 1, \dots, d.$$

This is a valid state: $\sum_{i=0}^d \hat{x}_{r,i} = \phi_r^* + \sum_v \mathbb{E}[D_v] u_{vr}^* = 1$ by constraint (3). Since $u_{vr}^{\text{LAG}}(\hat{x}) = \lambda_v \text{waterfill}_r(k, \phi^*, L(v, \cdot)) u_{vr}^*$ by Lemma 4.3, we verify $G(\hat{x}) = \hat{x}$:

$$G_{r,0}(\hat{x}) = \hat{x}_{r,0} - \sum_v u_{vr}^* + \hat{x}_{r,1} = \phi_r^* - \sum_v u_{vr}^* + \sum_v u_{vr}^* = \phi_r^* = \hat{x}_{r,0},$$

$$G_{r,i}(\hat{x}) = \hat{x}_{r,i+1} + \sum_v \Pr(D_v = i) u_{vr}^* = \sum_v [\Pr(D_v \geq i+1) + \Pr(D_v = i)] u_{vr}^* = \hat{x}_{r,i}, \quad i \geq 1.$$

By Assumption 4.2 the fixed point is unique, so $x^* = \hat{x}$, giving $u^{\text{LAG}}(x^*) = u^*$. Therefore the achievability bound holds:

$$\frac{\text{Reward}_{m,n}(\text{LAG})}{mn} \rightarrow \sum_{v,r} s_{vr} u_{vr}^* = \text{OPT}(\text{OCC-LP}(0)).$$

For the matching upper bound, fix any work-conserving policy π . Applying Proposition 3.2 to the (m, n) -scaled instance gives $\text{Reward}_{m,n}(\pi)/mn \leq \text{OPT}(\text{OCC-LP}(B_{m,n}))$, where $B_{m,n}$ is the blocking probability of the scaled system. Blocking is exactly the event of Proposition 4.7, an arriving task finding fewer than k free servers, so $B_{m,n} = P_{m,n}^{\text{block}} \rightarrow 0$.

It remains to show $\text{OPT}(\text{OCC-LP}(B_{m,n})) \rightarrow \text{OPT}(\text{OCC-LP}(0))$. Write $V(b)$ for the optimal value of the LP obtained by replacing the demand right-hand side in (2) with a parameter $b \in \mathbb{R}^V$, so that $\text{OPT}(\text{OCC-LP}(B)) = V(k\lambda(1-B))$, where $k\lambda(1-B) = (k\lambda_v(1-B))_{v \in [V]}$. The parameter b enters only the constraint right-hand sides, so V is concave and finite on the set of b for which the LP is feasible, hence continuous on the interior of that set. Strict underload (Assumption 4.1) gives $R(1-\rho) > k$, so $\text{OCC-LP}(0)$ admits a feasible point with strict slack in every availability constraint (4); perturbing b near $k\lambda$ preserves feasibility, so $b = k\lambda$ lies in the interior. Since $k\lambda(1-B_{m,n}) \rightarrow k\lambda$ as $B_{m,n} \rightarrow 0$, continuity of V at the interior point $k\lambda$ gives $\text{OPT}(\text{OCC-LP}(B_{m,n})) \rightarrow \text{OPT}(\text{OCC-LP}(0))$.

Hence $\limsup_{m,n \rightarrow \infty} \text{Reward}_{m,n}(\pi)/mn \leq \text{OPT}(\text{OCC-LP}(0))$, matching the achievability bound, and LAG is asymptotically optimal. $\square$

C Full Load-Sweep Results

Table 3 reports reward as a percentage of OCC-UB for all policies across the full ρ sweep, for each of the six instances.

Table 3. Reward as % of OCC-UB across load ρ , all instances.

Instance	ρ	LAG	Whittle	ALG-SC-LP	Greedy	Random
TMLR	0.1	98.9	79.3	74.8	98.2	36.5
	0.2	98.1	82.7	78.4	95.8	38.8
	0.3	97.1	84.8	76.2	92.6	40.9
	0.4	94.6	85.2	85.2	88.2	43.1
	0.5	92.8	85.6	83.0	84.7	45.2
	0.6	90.8	85.2	81.6	81.6	47.4
	0.7	87.8	83.2	80.5	78.4	49.6
	0.8	85.8	82.3	79.4	76.4	51.3
	0.9	85.5	82.8	78.7	76.5	53.5
ICLR	0.1	98.8	88.0	87.7	98.3	55.6
	0.2	98.2	89.5	90.2	96.8	58.3
	0.3	98.1	91.4	89.7	95.4	61.4
	0.4	97.0	92.1	91.8	93.0	64.3
	0.5	96.6	93.1	91.9	91.6	67.3

continued on next page

Instance	ρ	LAG	Whittle	ALG-SC-LP	Greedy	Random
	0.6	95.8	93.1	91.8	90.3	70.0
	0.7	94.5	92.5	90.0	88.9	72.0
	0.8	93.6	92.1	89.6	87.9	74.1
	0.9	93.1	92.3	89.0	87.8	75.1
Specialist	0.1	98.8	96.9	95.9	96.9	37.8
	0.2	94.5	90.4	90.2	90.4	37.7
	0.3	88.7	83.7	84.9	83.7	37.9
	0.4	89.1	83.4	86.4	83.4	41.5
	0.5	88.7	82.8	86.7	82.8	45.5
	0.6	88.7	83.6	86.8	83.6	49.1
	0.7	90.6	85.5	84.6	85.5	55.0
	0.8	91.9	87.4	85.1	87.4	61.5
Priority	0.1	88.6	83.5	89.2	88.6	41.1
	0.2	87.8	87.8	80.8	81.4	44.0
	0.3	87.8	87.8	77.5	73.5	46.4
	0.4	86.0	86.0	76.5	68.0	47.6
	0.5	86.2	86.2	77.3	65.1	50.6
	0.6	86.0	86.0	79.6	64.0	53.0
	0.7	85.6	85.6	82.2	63.0	54.9
	0.8	83.0	83.0	82.9	62.7	56.8
Unfriendly	0.1	98.1	98.1	97.5	97.5	26.6
	0.2	93.7	93.7	92.5	90.0	27.4
	0.3	89.3	89.3	86.1	80.6	28.7
	0.4	88.8	88.8	84.0	75.4	32.2
	0.5	89.2	89.2	83.6	70.9	35.5
	0.6	89.7	89.7	84.7	69.1	39.4
	0.7	88.7	88.7	82.2	66.8	42.7
	0.8	87.8	87.8	80.0	65.2	44.7
	0.9	85.3	85.3	78.1	63.3	47.7
Low-rank	0.1	98.8	43.0	96.0	98.4	51.7
	0.2	97.8	47.5	95.6	96.6	54.6
	0.3	96.1	51.1	94.0	94.0	55.8
	0.4	95.5	55.8	93.5	92.3	58.6
	0.5	95.4	61.0	92.6	90.7	61.1
	0.6	94.6	66.2	92.0	88.5	63.7
	0.7	94.3	71.9	91.8	86.3	67.1
	0.8	93.0	76.6	91.1	83.6	69.7
	0.9	91.4	80.0	89.6	81.6	71.9